

КОНСОЛИДИРОВАННОЕ ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Платформа БайЭл v3.0 (ФИНАЛЬНАЯ ВЕРСИЯ)

****Дата создания:**** 28 января 2026

****Версия:**** 3.0 Final

****Статус:**** Утверждено к разработке

****Основано на:**** Анализ всех ТЗ + Код v2.0 + Дизайн Figma + Уточнения от клиента



СОДЕРЖАНИЕ

1. [Критические изменения v3.0](#)
 2. [Описание бизнес-модели](#)
 3. [Роли и права доступа](#)
 4. [Аутентификация](#)
 5. [Система магазинов](#)
 6. [Каталог товаров](#)
 7. [Партнёрский инвентарь](#)
 8. [Workflow заказов](#)
 9. [Система долгов](#)
 10. [Бонусная система](#)
 11. [Возвращённые товары](#)
 12. [Бракованные товары](#)
 13. [Система расходов](#)
 14. [Журнал производства](#)
 15. [Себестоимость и наценка](#)
 16. [Статистика](#)
 17. [География](#)
 18. [Чаты и уведомления](#)
 19. [API Endpoints](#)
 20. [Модели данных](#)
-

1. КРИТИЧЕСКИЕ ИЗМЕНЕНИЯ v3.0

✅ ПОДТВЕРЖДЕНО (финальная версия)

1.1 Owner-based доступ к магазинам

- Каждый магазин принадлежит своему создателю (`store.owner = user`)
- Один владелец может создать ****несколько магазинов****
- Пользователь видит ****только свои магазины****
- Доступ контролируется через `IsStoreOwner` permission

1.2 Партнёрский инвентарь существует

- ****PartnerInventory**** - инвентарь партнёра
- ****PartnerRequest**** - запросы партнёра на товары у админа
- Партнёр запрашивает товары → Админ одобряет → Товары в PartnerInventory
- Партнёр использует товары для ****ручного сбора****

1.3 Workflow заказов - НОВАЯ ЛОГИКА

****Предзаказ (от магазина):****

Магазин создаёт → PENDING

→ Партнёр принимает → IN_TRANSIT (может возвращать товары)

→ Партнёр подтверждает → ACCEPTED (товары в StoreInventory, долг создан)

****Ручной сбор (от партнёра):****

Партнёр создаёт заказ → ACCEPTED (сразу, минуя PENDING и IN_TRANSIT)

****КРИТИЧНО:**** Админ НЕ участвует в одобрении заказов магазинов!

- Заказы идут НАПРЯМУЮ от магазина к партнёру
- Админ видит заказы только для МОНИТОРИНГА

1.4 Товары в инвентарь магазина

Товары попадают в **StoreInventory** только при **подтверждении партнёром** (статус ACCEPTED)

1.5 Долг создаётся при подтверждении партнёром

```
долг = total_amount - prepayment - returned_items_amount
```

1.6 Бонусы - кумулятивно по инвентарю

- Каждый 21-й товар бесплатно
- Считается по **суммарному количеству в StoreInventory**
- Только для **штучных товаров**

1.7 Система расходов (Сюзерен/Вассал/Обыватель)

- Используется для расчёта себестоимости
- Админ настраивает в разделе "Себестоимость"
- Интеграция с "Журналом производства"

1.8 Журнал производства (НОВОЕ)

- Админ добавляет партии товаров массово (50,000 - 200,000 шт)
- Система автоматически рассчитывает себестоимость партии
- Считает по "мешкам" ингредиентов

1.9 Возвращённые товары (ReturnedItem)

- Партнёр отмечает товары для возврата при обработке предзаказа
- Товары **НЕ попадают** в StoreInventory
- Отображаются в истории магазина

1.10 Деактивация магазина

- Магазин **НЕ может** создавать новые заказы
- Магазин **МОЖЕТ** входить в систему
- Отображается в списке с ограничениями

2. ОПИСАНИЕ БИЗНЕС-МОДЕЛИ

2.1 Цепочка поставок



↓

Конечный потребитель

2.2 Финансовые потоки

Админ

↓ запрос товаров (PartnerRequest)

Партнёр (PartnerInventory)

↓ продажа (StoreOrder)

Магазин (StoreInventory)

↓ долг создаётся

Погашение → Доход партнёра

Расходы партнёра → Видны админу

2.3 Ключевые бизнес-процессы

****Админ:****

- Управляет общим каталогом товаров
- Одобряет запросы партнёров на товары
- Видит расходы всех партнёров
- Рассчитывает себестоимость и наценку
- Мониторит заказы магазинов (без одобрения)
- Управляет географией (области, города)

****Партнёр:****

- Запрашивает товары у админа
- Принимает предзаказы от магазинов
- Создаёт ручные заказы магазинам
- Отмечает товары для возврата

- Управляет долгами магазинов
- Ведёт учёт своих расходов

****Магазин:****

- Создаёт предзаказы (выбирает из каталога админа)
- Погашает долги
- Просматривает историю и инвентарь
- Заявляет о бракованных товарах

3. РОЛИ И ПРАВА ДОСТУПА

3.1 Администратор (Admin)

****Функции:****

- ☒ CRUD каталога товаров
- ☒ Журнал производства (массовое добавление)
- ☒ Система расходов (Сюзерен/Вассал/Обыватель)
- ☒ Себестоимость и наценка
- ☒ Одобрение/отклонение запросов партнёров
- ☒ ****Мониторинг заказов магазинов**** (НЕ одобрение!)
- ☒ Управление областями и городами
- ☒ Активация/деактивация магазинов
- ☒ Просмотр глобальной статистики

****Permissions:****

```
IsAdmin: user.role == 'admin'
```

3.2 Партнёр (Partner)

****Функции:****

- ☒ Управление PartnerInventory
- ☒ Запросы товаров (PartnerRequest)
- ☒ Приём предзаказов (PENDING → IN_TRANSIT → ACCEPTED)
- ☒ Создание ручных заказов

- ☒ Отметка возвращённых товаров
- ☒ Указание предоплаты
- ☒ Учёт расходов (PartnerExpense)
- ☒ Приём погашений долгов
- ☒ Подтверждение брака

****Permissions:****

```
IsPartner: user.role == 'partner'
```

```
CanAccessPartnerInventory: inventory.partner == user
```

3.3 Магазин (Store)

****Функции:****

- ☒ Создание предзаказов
- ☒ Просмотр собственного инвентаря
- ☒ Погашение долгов
- ☒ Просмотр истории
- ☒ Заявка о бракованных товарах

****Ограничения:****

- ☒ Видит только свои магазины (owner=user)
- ☒ Не может удалять товары из инвентаря
- ☒ Не может создавать заказы при деактивации

****Permissions:****

```
IsStore: user.role == 'store'
```

```
IsStoreOwner: store.owner == user
```

4. АУТЕНТИФИКАЦИЯ

4.1 Скрытая валидация роли через маркер

****Механизм:****

```
# При регистрации

password = request.data['password']

marker = settings.PARTNER_MARKER # "p!8Rt"

if marker in password:

    role = 'partner'

    password = password.replace(marker, '')

else:

    role = 'store'

user.set_password(password) # Маркер удалён!

user.role = role

user.save()
```

****Требования к маркеру:****

- Длина: 3-5 символов
- Уникальный: p!8Rt (пример)
- Хранится в .env : PARTNER_MARKER=p!8Rt
- Вставляется в произвольной позиции
- НЕ сохраняется в БД

****Примеры:****

```
"12345p!8Rt67890" → role='partner', password='1234567890'
```



```
"mypass123" → role='store', password='mypass123'
```

4.2 Регистрация

****Endpoint:****

```
POST /api/auth/register/
```

****Магазин:****

```
{  
  
  "phone": "+996777123456",  
  
  "email": "store@example.com",  
  
  "password": "mypassword123",  
  
  "name": "Иван",  
  
  "second_name": "Иванов"  
}
```

****Партнёр:****

```
{  
  
  "phone": "+996777123456",  
  
  "email": "partner@example.com",  
  
  "password": "myp!8Rtassword123", // Маркер в пароле!  
  
  "name": "Петр",  
  
  "second_name": "Петров"
```

```
}
```

****Response:****

```
{  
  
  "access": "eyJ0eXAiOiJKV...",  
  
  "refresh": "eyJ0eXAiOiJKV...",  
  
  "user": {  
  
    "id": 1,  
  
    "phone": "+996777123456",  
  
    "email": "partner@example.com",  
  
    "role": "partner"  
  
  }  
  
}
```

4.3 Вход (Login)

****Endpoint:****

```
POST /api/auth/login/
```

****Body:****

```
{  
  
  "phone": "+996777123456", // или email  
  
  "password": "mypassword123",  
  
}
```

```
"remember_me": true  
  
}
```

****Логика:****

- `remember_me=false` : Access 60 мин, Refresh 7 дней
- `remember_me=true` : Access 90 дней, Refresh 180 дней

4.4 Сброс пароля (3 этапа)

****Этап 1: Запрос кода****

POST /api/auth/password/reset/

Body: { "email": "user@example.com" }

Процесс:

1. Проверка существования email
2. Генерация 5-значного кода (12345)
3. Отправка на email
4. Код действителен 15 минут

****Этап 2: Верификация****

POST /api/auth/password/verify/

Body: { "email": "user@example.com", "code": "12345" }

Response: { "token": "temp_token_for_reset" }

****Этап 3: Установка****

POST /api/auth/password/confirm/

Body: { "token": "...", "new_password": "newpass123" }

Результат:

- Пароль изменён
- Все старые токены инвалидированы

4.5 Валидация полей

****Email:****

`^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$`

Длина: ≤ 50 символов

Один символ @

Уникальный

****Phone:****

`^\+996[0-9]{9}$`

Формат: +996XXXXXXXXX

Уникальный

USERNAME_FIELD

5. СИСТЕМА МАГАЗИНОВ

5.1 Owner-based модель (v3.0)

****Принцип:****

```
# Каждый магазин принадлежит создателю

store = Store.objects.create(

    name="Магазин Ак-мет",

    owner=request.user, # Владелец!

    ...

)

# Пользователь видит только свои

my_stores = Store.objects.filter(owner=request.user)
```

****Правила:****

-  Один пользователь → много магазинов
-  Пользователь может быть только в ****одном**** магазине одновременно
-  Пользователь может выйти и переключиться на другой свой магазин
-  Несколько пользователей ****НЕ МОГУТ**** быть в одном магазине (изменение от v2.0!)

****Permission:****

```
class IsStoreOwner(BasePermission):

    def has_object_permission(self, request, view, obj):

        return obj.owner == request.user
```

5.2 Регистрация магазина

****Endpoint:****

```
POST /api/stores/
```

****Body:****

```
{  
  
  "name": "Магазин Белес-Духонуу",  
  
  "owner_name": "Сыдыков Алихан Байаманов",  
  
  "phone": "+996774140282",  
  
  "inn": "35545465565",  
  
  "region": 1,  
  
  "city": 5,  
  
  "address": "Жапарова 145 Б"  
  
}
```

****Валидация:****

- name : уникальное
- inn : 12-14 символов, только цифры, уникальный
- phone : +996XXXXXXXXXX
- region и city : должны существовать

****Response:****

```
{  
  
  "id": 1,
```

```
"name": "Магазин Белес-Духонуу",  
  
"owner": 5,  
  
"owner_name": "Сыдыков Алихан Байаманов",  
  
"phone": "+996774140282",  
  
"inn": "35545465565",  
  
"region": { "id": 1, "name": "Ошская область" },  
  
"city": { "id": 5, "name": "Ош" },  
  
"address": "Жапарова 145 Б",  
  
"is_active": true,  
  
"total_debt": "0.00",  
  
"paid_debt": "0.00"  
  
}
```

5.3 Статусы магазина

****Активный (**is_active=True**):****

-  Может создавать заказы
-  Может погашать долги
-  Полный функционал

****Деактивированный (**is_active=False**):****

-  НЕ может создавать новые заказы
-  МОЖЕТ входить в систему
-  НЕ может погашать долги
-  Отображается в списке с пометкой

****Деактивация:****

```
POST /api/admin/stores/{id}/deactivate/
```

****Активация:****

```
POST /api/admin/stores/{id}/activate/
```

5.4 Фильтрация и поиск

****Фильтры (Admin, Partner):****

- По области
- По городу
- ****Долг**** - магазины с `total_debt > 0` (сортировка по убыванию)
- ****Погашен**** - магазины с `total_debt = 0` (сортировка по дате погашения)

****Поиск:****

- По ИНН (12-14 символов)
- По названию
- По городу

****Endpoint:****

```
GET /api/stores/?region=1&city=5&debt_status=has_debt&search=Белес
```

6. КАТАЛОГ ТОВАРОВ

6.1 Типы товаров

****Штучные (Piece-based):****

```
is_weight_based = False  
  
unit = "шт"
```



```
price = 200.00 # за 1 шт
```

****Особенности:****

-  Могут быть бонусными
-  Правило "каждый 21-й бесплатно"
-  Примеры: пельмени, хинкали, вареники

****Весовые (Weight-based):****

```
is_weight_based = True  
  
unit = "кг"  
  
price = 450.00 # за 1 кг
```

****Особенности:****

-  НЕ могут быть бонусными
-  Минимум: 1 кг (или 0.1 кг если остаток < 1 кг)
-  Шаг: 0.1 кг (100 грамм)
-  Могут быть бракованными
-  Примеры: курица, мука, фарш

6.2 Расчёт цены весовых товаров

****Формула:****

```
цена_за_100г = цена_за_1кг ÷ 10  
  
итоговая_цена = цена_за_100г × (вес ÷ 0.1)
```

****Примеры:****

```
# Курица: 450 сом/кг, вес 2.5 кг  
  
price_100g = 450 ÷ 10 = 45 сом
```

```
total = 45 × (2.5 ÷ 0.1) = 45 × 25 = 1,125 сом
```

```
# Мука: 50 сом/кг, вес 10 кг
```

```
price_100g = 50 ÷ 10 = 5 сом
```

```
total = 5 × (10 ÷ 0.1) = 5 × 100 = 500 сом
```

6.3 Модель Product

```
class Product(models.Model):

    name = models.CharField(max_length=255, unique=True)

    price = models.DecimalField(max_digits=10, decimal_places=2)

    quantity = models.DecimalField(max_digits=10, decimal_places=2)

    is_weight_based = models.BooleanField(default=False)

    is_bonus = models.BooleanField(default=False)

    description = models.TextField(max_length=250)

    is_active = models.BooleanField(default=True)

    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:

        constraints = [

            models.CheckConstraint(

                check=~models.Q(is_weight_based=True, is_bonus=True),

                name='weight_based_cannot_be_bonus'

            )
```

]

6.4 CRUD операции (только Admin)

****Создание:****

```
POST /api/admin/products/
```

```
Content-Type: multipart/form-data
```

Body:

```
- name: "Пельмени ХУТОРОК 900г"
- price: 200.00
- quantity: 537
- is_weight_based: false
- is_bonus: false
- description: "Пельмени домашнего..."
- images: [file1.jpg, file2.jpg, file3.jpg]
```

****Валидация:****

- Название уникальное
- 1-3 изображения (JPG/PNG, до 5 МБ)
- Описание до 250 символов
- ❌ Весовые НЕ могут быть бонусными

****Редактирование:****

```
PUT /api/admin/products/{id}/
```

Можно изменить:

- ✓ Цену
- ✓ Название
- ✓ Описание
- ✓ Изображения
- ✓ Статус бонусный/обычный

✗ НЕЛЬЗЯ изменить `is_weight_based` (весовой → штучный)

****Удаление:****

```
DELETE /api/admin/products/{id}/
```

Подтверждение через модальное окно:

"Вы уверены, что хотите удалить товар?"

[Нет] [Да]

Результат:

- Товар пропадает из каталога
- Партнёры не могут его продавать
- Проданные товары остаются в истории
- Остатки на складе исчезают

****Добавление на склад:****

```
POST /api/admin/products/{id}/add-stock/
```

```
Body: { "quantity": 100 }
```

```
Результат: product.quantity += 100
```

7. ПАРТНЁРСКИЙ ИНВЕНТАРЬ

7.1 Модель PartnerInventory

```
class PartnerInventory(models.Model):

    partner = models.ForeignKey(User, on_delete=models.CASCADE)

    product = models.ForeignKey(Product, on_delete=models.CASCADE)

    quantity = models.DecimalField(max_digits=10, decimal_places=2)

    weight = models.DecimalField(max_digits=10, decimal_places=2, null=True)

    partner_request_id = models.ForeignKey(PartnerRequest, null=True)

    class Meta:

        unique_together = ['partner', 'product']
```

7.2 Запросы товаров (PartnerRequest)

****Типы запросов:****

```
REQUEST_TYPE_CHOICES = [

    ('request', 'Запрос товаров'),

    ('return', 'Возврат товаров')
```

```
]
```

****Статусы:****

```
STATUS_CHOICES = [  
    ('pending', 'В ожидании'),  
    ('approved', 'Одобрено'),  
    ('rejected', 'Отклонено')  
]
```

****Модель:****

```
class PartnerRequest(models.Model):  
    partner = models.ForeignKey(User)  
    product = models.ForeignKey(Product)  
    request_type = models.CharField(choices=REQUEST_TYPE_CHOICES)  
    status = models.CharField(choices=STATUS_CHOICES, default='pending')  
    quantity = models.DecimalField()  
    reason = models.TextField(blank=True)  
    reviewed_by = models.ForeignKey(User, null=True,  
related_name='reviewed_requests')  
    reviewed_at = models.DateTimeField(null=True)  
    rejection_reason = models.TextField(blank=True)  
    created_at = models.DateTimeField(auto_now_add=True)
```

7.3 Workflow запроса товаров (REQUEST)



****Endpoint (Partner):****

POST /api/partners/requests/

```
Body: {  
  
  "products": [  
  
    {"product": 5, "quantity": 100},  
  
    {"product": 8, "quantity": 50, "weight": "25.0"}  
  
  ],  
  
  "reason": "Запрос на 900 шт 20кг"  
  
}
```

****Endpoint (Admin - одобрение):****

POST /api/admin/partner-requests/{id}/approve/

Процесс:

1. status = 'approved'
2. Product.quantity -= request.quantity
3. PartnerInventory.quantity += request.quantity
4. reviewed_by = request.user
5. reviewed_at = now()
6. Уведомление партнёру

****Endpoint (Admin - отклонение):****

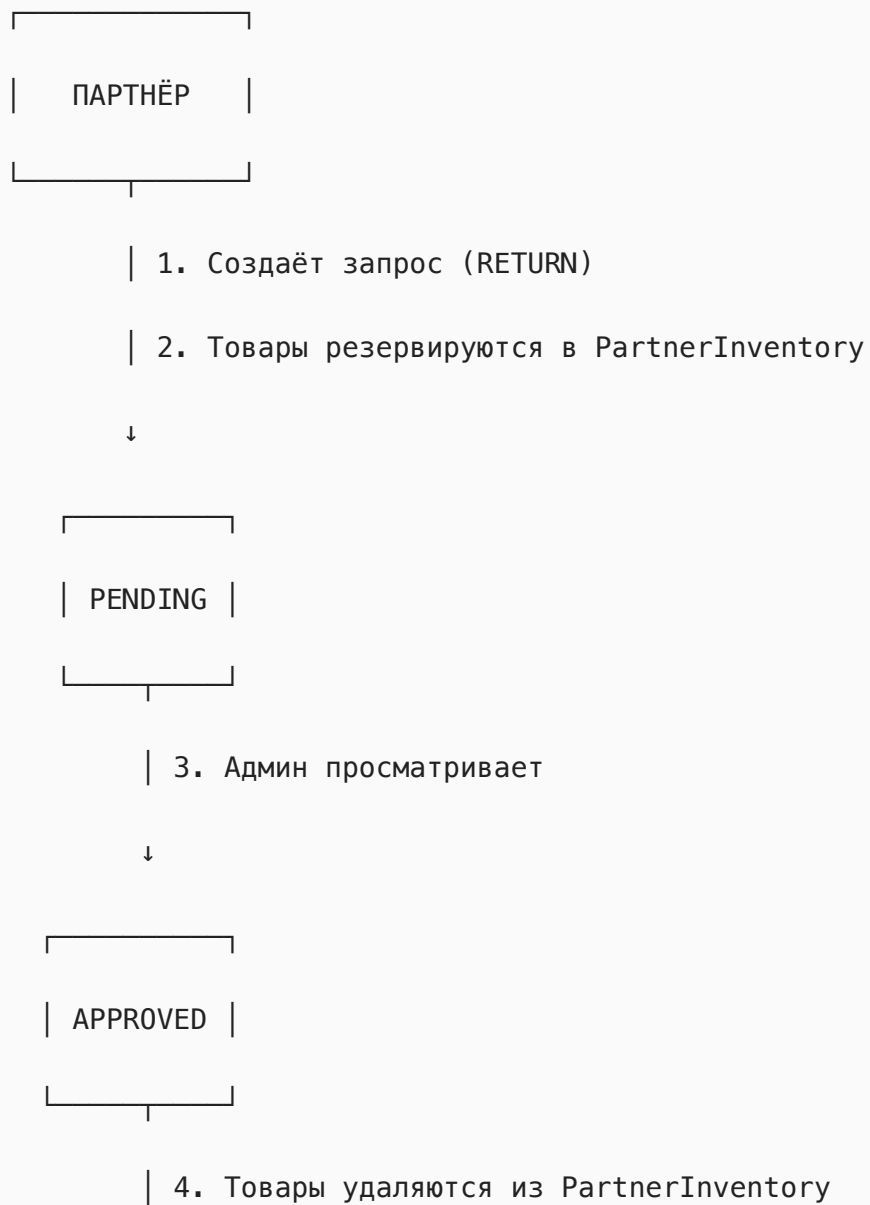
POST /api/admin/partner-requests/{id}/reject/

Body: { "rejection_reason": "Недостаточно товара на складе" }

Процесс:

1. `status = 'rejected'`
2. `reviewed_by = request.user`
3. `reviewed_at = now()`
4. Уведомление партнёру

7.4 Workflow возврата товаров (RETURN)



| 5. Товары возвращаются в каталог админа

↓

```
PartnerInventory.quantity -= request.quantity
```

```
Product.quantity += request.quantity
```

****Endpoint (Partner):****

```
POST /api/partners/requests/
```

```
Body: {
```

```
  "request_type": "return",
```

```
  "products": [
```

```
    {"product": 5, "quantity": 50}
```

```
  ],
```

```
  "reason": "Лишние товары"
```

```
}
```

8. WORKFLOW ЗАКАЗОВ (КРИТИЧЕСКАЯ СЕКЦИЯ!)

8.1 Два типа заказов

****Предзаказ (Preorder):****

- Инициатива: ****Магазин****
- Источник товаров: ****Каталог админа****
- Статусы: PENDING → IN_TRANSIT → ACCEPTED/REJECTED

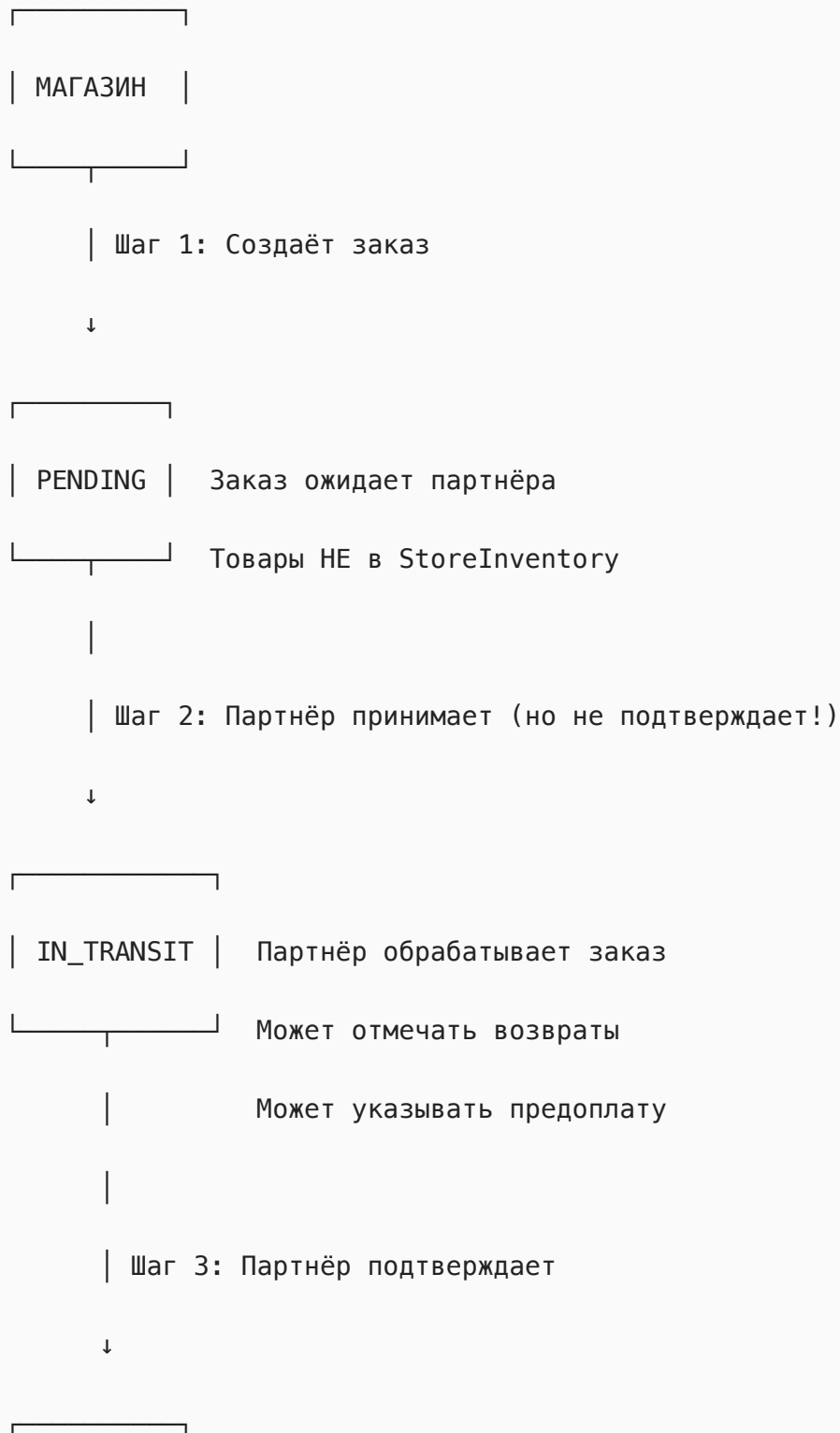
****Ручной сбор (Manual):****

- Инициатива: ****Партнёр****

- Источник товаров: ****PartnerInventory****
- Статусы: Сразу ACCEPTED

8.2 Предзаказ - Полный цикл

****Диаграмма:****



| ACCEPTED | Товары → StoreInventory

| Долг создаётся

****Шаг 1: Магазин создаёт предзаказ****

Endpoint:

POST /api/stores/orders/

Body:

```
{
  "items": [
    {
      "product": 5,
      "quantity": 20
    },
    {
      "product": 8,
      "quantity": 1,
      "weight": "2.5"
    }
  ]
}
```

Процесс:

1. Валидация: магазин активен?
2. Создание StoreOrder (status=PENDING, order_type='preorder')
3. Создание StoreOrderItem для каждого товара
4. Расчёт total_amount
5. Уведомление партнёру: "Новый предзаказ от магазина X"

Response:

```
{  
  "id": 1,  
  "store": {  
    "id": 1,  
    "name": "Магазин Ак-мет"  
  },  
  "status": "pending",  
  "order_type": "preorder",  
  "total_amount": "2200.00",  
  "items": [  
    {  
      "product": {  
        "id": 5,  
        "name": "Пельмени красные",  
        "price": "200.00"
```

```
    },  
  
    "quantity": "20.00",  
  
    "total_price": "4000.00"  
  },  
  
  {  
  
    "product": {  
  
      "id": 8,  
  
      "name": "Курица",  
  
      "price": "450.00"  
    },  
  
    "weight": "2.50",  
  
    "total_price": "1125.00"  
  }  
],  
  
  "created_at": "2025-05-28T10:00:00Z"  
}
```

****Шаг 2: Партнёр принимает заказ****

Endpoint:

```
POST /api/partners/preorders/{id}/accept/
```

Процесс:

1. Валидация: статус = PENDING?

2. Обновление: `status = IN_TRANSIT`
3. `confirmed_by = request.user` (партнёр)
4. Уведомление магазину: "Ваш заказ принят партнёром"

Response:

```
{  
  "id": 1,  
  "status": "in_transit",  
  "message": "Заказ принят. Теперь вы можете обработать товары."  
}
```

****Шаг 3: Партнёр подтверждает заказ****

Endpoint:

```
POST /api/partners/preorders/{id}/confirm/
```

Body:

```
{  
  "returned_items": [  
    {  
      "product": 5,  
      "quantity": 5,  
      "reason": "Нет в наличии"    }  
  ]  
}
```

```
    }  
  
    ],  
  
    "prepayment": "500.00"  
  
}
```

Процесс:

1. Валидация: статус = IN_TRANSIT?
2. Создание ReturnedItem для каждого возвращённого товара
3. Товары из PartnerInventory → StoreInventory (за вычетом возвратов)
4. Расчёт бонусов (если штучные)
5. Долг = total_amount - prepayment - returned_items_amount
6. status = ACCEPTED
7. accepted_at = now()
8. Уведомление магазину: "Ваш заказ подтверждён"

Пример расчёта:

Исходный заказ

Пельмени: 20 шт × 200 = 4,000 сом

Курица: 2.5 кг × 450 = 1,125 сом

Итого: 5,125 сом

Возвраты

Пельмени: 5 шт × 200 = 1,000 сом

Предоплата

500 сом

Долг

5,125 - 1,000 - 500 = 3,625 сом

Инвентарь магазина

Пельмени: 15 шт (20 - 5 возврат)

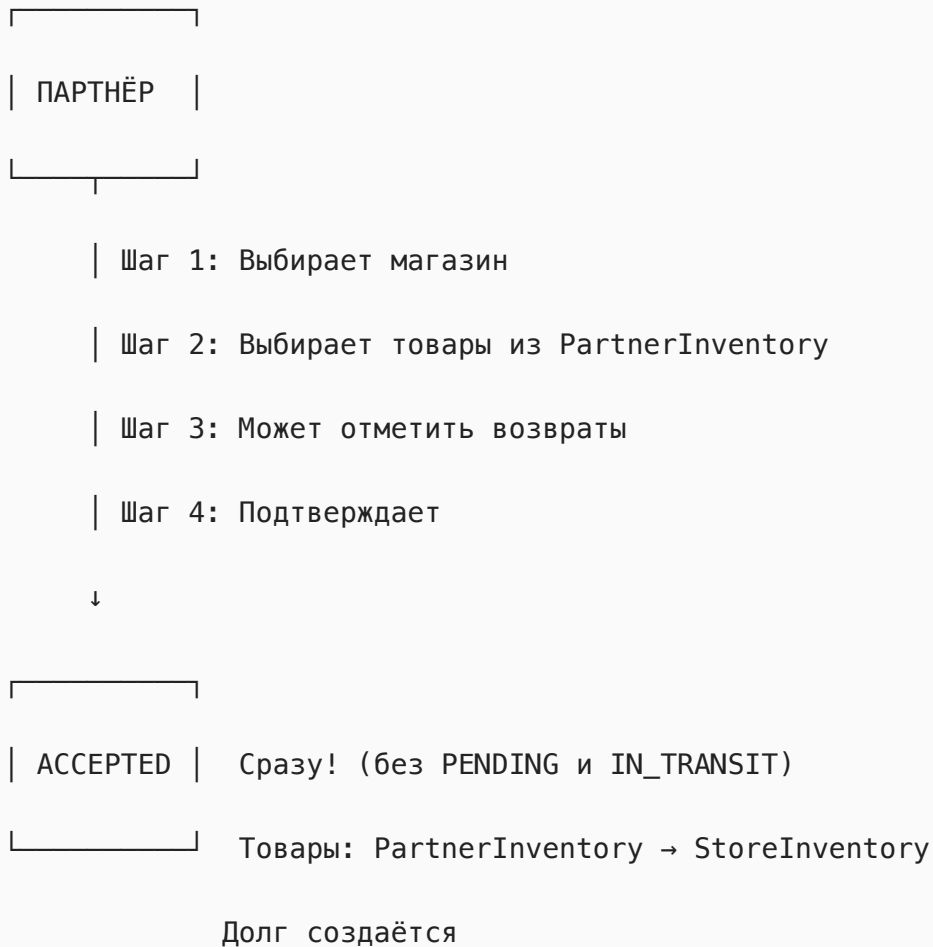
Курица: 2.5 кг

Response:

```
{
  "id": 1,
  "status": "accepted",
  "total_amount": "5125.00",
  "returned_amount": "1000.00",
  "prepayment": "500.00",
  "debt_amount": "3625.00",
  "accepted_at": "2025-05-28T12:00:00Z"
}
```

8.3 Ручной сбор - Полный цикл

****Диаграмма:****



****Endpoint:****

```
POST /api/partners/manual-orders/
```

****Body:****

```
{  
  
  "store": 1,  
  
  "items": [  

```

```
{
  "product": 5,
  "quantity": 30
},
{
  "product": 8,
  "quantity": 1,
  "weight": "5.0"
}
],
"returned_items": [
  {
    "product": 5,
    "quantity": 5,
    "reason": "Бракованные"
  }
],
"prepayment": "1000.00"
}
```

****Процесс:****

1. Валидация:

- Магазин существует и активен?

- Товары есть в PartnerInventory?
- Достаточно количества?

2. Списание из PartnerInventory

3. Добавление в StoreInventory (за вычетом возвратов)

4. Создание ReturnedItem (если есть)

5. Расчёт долга:

```
debt = total_amount - prepayment - returned_items_amount
```

6. Создание StoreOrder:

- status = ACCEPTED
- order_type = 'manual'
- confirmed_by = request.user

7. Уведомление магазину: "Партнёр передал вам товары"

****Response:****

```
{  
  
  "id": 2,  
  
  "store": {
```

```
    "id": 1,

    "name": "Магазин Ак-мет"

},

    "status": "accepted",

    "order_type": "manual",

    "total_amount": "8250.00",

    "returned_amount": "1000.00",

    "prepayment": "1000.00",

    "debt_amount": "6250.00",

    "created_at": "2025-05-28T14:00:00Z",

    "accepted_at": "2025-05-28T14:00:00Z"

}
```

8.4 Роль админа в заказах

****КРИТИЧНО:**** Админ ****НЕ одобряет**** заказы магазинов!

Админ видит заказы в разделе ****"Запросы → Магазины"***** только для:

- ****Мониторинга****
- ****Статистики****
- ****Анализа****

****Endpoint (Admin):****

```
GET /api/admin/store-orders/
```

****Response:****

```
{  
  
  "count": 150,  
  
  "results": [  
  
    {  
  
      "id": 1,  
  
      "store": {  
  
        "name": "Магазин Ак-мет",  
  
        "inn": "123456789012"  
  
      },  
  
      "partner": {  
  
        "name": "Зргешов Тыныктык"  
  
      },  
  
      "status": "accepted",  
  
      "order_type": "preorder",  
  
      "total_amount": "5125.00",  
  
      "created_at": "2025-05-28T10:00:00Z"  
  
    }  
  
  ]  
  
}
```

9. СИСТЕМА ДОЛГОВ

9.1 Создание долга

****Момент создания:****

- При подтверждении партнёром (status = ACCEPTED)

****Формула:****

```
debt = total_amount - prepayment - returned_items_amount
```

****Обновление Store:****

```
store.total_debt += debt  
  
store.save()
```

****Примеры:****

Пример 1: Полная оплата

```
total = 5,000  
  
prepayment = 5,000  
  
returned = 0  
  
→ debt = 0 # Нет долга
```

Пример 2: Частичная предоплата

```
total = 5,000  
  
prepayment = 2,000  
  
returned = 0  
  
→ debt = 3,000
```

Пример 3: С возвратами

```
total = 5,000  
  
prepayment = 500  
  
returned = 1,500  
  
→ debt = 3,000
```

Пример 4: Переплата

```
total = 5,000  
  
prepayment = 6,000  
  
returned = 0  
  
→ debt = -1,000 # Переплата (отрицательный долг)
```

9.2 Погашение долга

****Модель DebtPayment:****

```
class DebtPayment(models.Model):  
  
    order = models.ForeignKey(StoreOrder, on_delete=models.CASCADE)  
  
    amount = models.DecimalField(max_digits=10, decimal_places=2)  
  
    paid_by = models.ForeignKey(User, related_name='debt_payments')  
  
    received_by = models.ForeignKey(User, related_name='received_payments')  
  
    comment = models.TextField(blank=True)  
  
    created_at = models.DateTimeField(auto_now_add=True)
```


****Endpoint:****

```
POST /api/stores/orders/{id}/pay-debt/
```

****Body:****

```
{  
  "amount": "1500.00",  
  "comment": "Частичное погашение"  
}
```

****Процесс:****

1. Валидация:

- Заказ существует?
- amount > 0?
- amount <= order.debt_amount?

2. Создание DebtPayment:

- paid_by = request.user (магазин)
- received_by = order.confirmed_by (партнёр)

3. Обновление заказа:

```
order.debt_amount -= amount
```

4. Обновление магазина:

```
store.total_debt -= amount
```

```
store.paid_debt += amount
```

5. Если долг полностью погашен:

– Уведомление: "Долг по заказу #{id} погашен"

6. Погашенный долг идёт в доход партнёра

****Response:****

```
{  
  
  "order_id": 1,  
  
  "paid_amount": "1500.00",  
  
  "remaining_debt": "2125.00",  
  
  "message": "Долг частично погашен"  
  
}
```

9.3 Типы погашения

****Частичное:****

```
amount < debt
```

```
→ debt -= amount
```

****Полное:****

```
amount == debt
```

```
→ debt = 0
```

```
→ Статус долга: "Погашен"
```

****Переплата:****

```
amount > debt
```

```
→ debt = debt - amount # Отрицательный
```

```
→ Отображается как кредит
```

10. БОНУСНАЯ СИСТЕМА

10.1 Правило "Каждый 21-й бесплатно"

****Формула:****

```
bonus_count = total_quantity ÷ 21 # целочисленное деление
```

```
paid_count = total_quantity - bonus_count
```

****Примеры:****

```
20 шт → bonus: 0, paid: 20 # Недостаточно для бонуса
```

```
21 шт → bonus: 1, paid: 20
```

```
40 шт → bonus: 1, paid: 39
```

42 шт → bonus: 2, paid: 40

100 шт → bonus: 4, paid: 96

10.2 Кумулятивный характер

Бонусы считаются по ****суммарному количеству в StoreInventory****:

Заказ #1

15 шт Пельменей → StoreInventory: 15 шт

→ bonus: 0 ($15 \div 21 = 0$)

Заказ #2

10 шт Пельменей → StoreInventory: 25 шт

→ bonus: 1 ($25 \div 21 = 1$)

→ paid: 24

Заказ #3

20 шт Пельменей → StoreInventory: 45 шт

→ bonus: 2 ($45 \div 21 = 2$)

→ paid: 43

****Обновление при добавлении:****

```
def update_bonus_count(inventory):  
    total_quantity = inventory.quantity  
  
    bonus_count = total_quantity // 21
```

```
paid_count = total_quantity - bonus_count

inventory.bonus_count = bonus_count

inventory.paid_count = paid_count

inventory.save()
```

10.3 Ограничения

****Бонусные товары:****

-  Только штучные (`is_weight_based=False`)
-  Весовые товары ****НЕ могут**** быть бонусными
-  Автоматический расчёт при добавлении в инвентарь
-  Админ может вручную пометить товар как бонусный (игнорирует правило 21)

****Валидация:****

```
if product.is_weight_based and product.is_bonus:

    raise ValidationError("Весовые товары не могут быть бонусными")
```

10.4 Влияние на статистику

****Бонусы в статистике:****

- Отображаются ****отдельно**** (количественно)
- ****НЕ влияют**** на сумму дохода
- Учитываются в общем количестве товаров
- Отображаются в истории магазина

****Пример в истории:****

Дата 24.04.2025

Общее количество товаров: 67

Пельмени красные 10 шт 200 сом

Курица сырая 30 кг 2000 сом

Бонус:

Пельмени красные 10 шт 200 сом

Долг: 2000 сом

11. ВОЗВРАЩЁННЫЕ ТОВАРЫ

11.1 Модель ReturnedItem

```
class ReturnedItem(models.Model):  
  
    order = models.ForeignKey(StoreOrder, on_delete=models.CASCADE)  
  
    product = models.ForeignKey(Product, on_delete=models.CASCADE)  
  
    quantity = models.DecimalField(max_digits=10, decimal_places=2)  
  
    weight = models.DecimalField(max_digits=10, decimal_places=2, null=True)  
  
    price = models.DecimalField(max_digits=10, decimal_places=2)  
  
    reason = models.TextField(blank=True)  
  
    returned_at = models.DateTimeField(auto_now_add=True)  
  
    returned_by = models.ForeignKey(User, on_delete=models.CASCADE)
```

11.2 Когда создаются

****Предзаказ:****

- Партнёр отмечает товары для возврата при подтверждении (IN_TRANSIT → ACCEPTED)
- Товары ****НЕ попадают**** в StoreInventory
- Уменьшают сумму заказа

****Ручной сбор:****

- Партнёр может отметить товары для возврата при создании заказа
- Принцип тот же

11.3 Пример

****Предзаказ:****

```
{  
  "order_id": 1,  
  "items": [  
    {"product": 5, "quantity": 20},  
    {"product": 8, "weight": "2.5"}  
  ],  
  "returned_items": [  
    {  
      "product": 5,  
      "quantity": 5,  
      "reason": "Нет в наличии"  
    }  
  ]  
}
```

```
}
```

****Результат:****

StoreInventory

Пельмени: 15 шт (20 – 5)

Курица: 2.5 кг

ReturnedItem

Пельмени: 5 шт (причина: "Нет в наличии")

Сумма заказа

$total = (20 \times 200) + (2.5 \times 450) = 5,125 \text{ сом}$

$returned = 5 \times 200 = 1,000 \text{ сом}$

→ итого к оплате: 4,125 сом

11.4 Отображение в истории

****История магазина:****

Дата 24.04.2025

Полученные товары:

– Пельмени красные 10 шт 200 сом

– Курица сырая 30 кг 2000 сом

Возвращённые товары:

- Пельмени красные 10 шт 1000 сом
- Пельмени красные 10 шт 1000 сом

Долг: 2000 сом

12. БРАКОВАННЫЕ ТОВАРЫ

12.1 Модель DefectiveProduct

```
class DefectiveProduct(models.Model):  
  
    store = models.ForeignKey(Store, on_delete=models.CASCADE)  
  
    product = models.ForeignKey(Product, on_delete=models.CASCADE)  
  
    order = models.ForeignKey(StoreOrder, on_delete=models.CASCADE,  
null=True)  
  
    quantity = models.DecimalField(max_digits=10, decimal_places=2)  
  
    weight = models.DecimalField(max_digits=10, decimal_places=2, null=True)  
  
    defect_amount = models.DecimalField(max_digits=10, decimal_places=2)  
  
    notes = models.TextField(blank=True)  
  
    reported_at = models.DateTimeField(auto_now_add=True)  
  
    confirmed_by = models.ForeignKey(User, on_delete=models.SET_NULL,  
null=True)
```

12.2 Workflow



1. Заявляет о браке



2. Партнёр просматривает



3. Подтверждает или отклоняет



Брак списывается



Заявка удаляется

(Ничего не происходит)

Долг уменьшается

```
StoreInventory.defective_count++
```

****Endpoint (Store):****

```
POST /api/stores/{store_id}/report-defective/
```

****Body:****

```
{  
  
  "product": 5,  
  
  "quantity": 3,  
  
  "order": 1,  
  
  "notes": "Упаковка порвана"  
  
}
```

****Endpoint (Partner - подтверждение):****

```
POST /api/partners/defective-products/{id}/confirm/
```

****Процесс:****

1. Валидация заявки
2. Расчёт суммы брака:

```
defect_amount = quantity × price
```

3. Уменьшение долга:

```
order.debt_amount -= defect_amount
```

```
store.total_debt -= defect_amount
```

4. Обновление инвентаря:

```
inventory.defective_count += quantity
```

```
inventory.quantity -= quantity
```

5. confirmed_by = request.user (партнёр)

6. Уведомление магазину: "Брак подтверждён"

****Endpoint (Partner - отклонение):****

```
POST /api/partners/defective-products/{id}/reject/
```

****Процесс:****

1. Удаление заявки

2. Уведомление магазину: "Заявка о браке отклонена"

12.3 Весовые товары

****Для весовых:****

```
# Курица: 450 сом/кг, брак 1.3 кг
```

```
defect_amount = 1.3 × 450 = 585 сом
```

```
# Фарш: 810 сом/кг, брак 0.5 кг
```

```
defect_amount = 0.5 × 810 = 405 сом
```

Документ стал очень большим! Продолжу в следующем файле со всеми остальными разделами (Расходы, Производство, Статистика, API, Модели). Сейчас сохраню текущий файл:

<function_calls>

["/home/claude/BAIEL_v3.0_FINAL_CONSOLIDATED_TS.md"]:

1. ****Owner-based доступ к магазинам****

- Каждый магазин принадлежит своему создателю (`store.owner = request.user`)
- Один владелец может создать несколько магазинов
- Пользователь видит только свои магазины
- Один пользователь может быть только в одном магазине одновременно
- Может выйти и переключиться на другой свой магазин

2. ****Партнёрский инвентарь существует****

- Партнёр запрашивает товары у админа (PartnerRequest)
- Админ одобряет → товары в PartnerInventory
- Партнёр использует инвентарь для создания заказов магазинам

3. ****Заказы магазина идут НАПРЯМУЮ к партнёру (минуя админа!)****

- ❌ СТАРОЕ: Магазин → Админ → Партнёр
- ✅ НОВОЕ: Магазин → Партнёр (напрямую)

- Админ видит заказы только для мониторинга

4. ****Два типа заказов:****

- ****Предзаказ (Preorder):**** инициатива магазина
 - PENDING → IN_TRANSIT → ACCEPTED/REJECTED
- ****Ручной сбор (Manual):**** инициатива партнёра
 - Сразу ACCEPTED (без промежуточных статусов)

5. ****Товары в инвентарь магазина - при подтверждении партнёром****

- PENDING → товары НЕ в инвентаре
- IN_TRANSIT → партнёр обрабатывает (может отмечать возвраты)
- ACCEPTED → товары в StoreInventory

6. ****Долг создаётся при подтверждении партнёром****

- Формула: $\text{долг} = \text{total_amount} - \text{prepayment} - \text{returned_items}$

7. ****Бонусы - кумулятивно по инвентарю магазина****

- Каждый 21-й товар бесплатно
- Считается по суммарному количеству в StoreInventory
- Только штучные товары

8. ****Возвращённые товары (ReturnedItem)****

- Партнёр отмечает товары для возврата при обработке заказа
- Товары НЕ попадают в инвентарь магазина
- Отображаются в истории магазина

9. ****Журнал производства (НОВОЕ!)****

- Админ добавляет партии товаров массово
- Считает себестоимость по партиям
- Система Сюзерен/Вассал/Обыватель для расходов

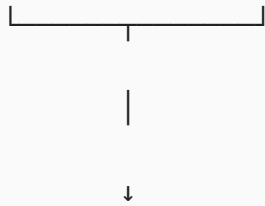
10. ****Деактивация магазина****

- Магазин не может создавать новые заказы
- Может входить в систему
- Не может погашать долги (требуется уточнения)

2. ОПИСАНИЕ БИЗНЕС-МОДЕЛИ

2.1 Цепочка поставок





Конечный потребитель

2.2 Финансовые потоки

Админ (каталог)

↓ запрос товаров

Партнёр (PartnerInventory)

↓ продажа

Магазин (StoreInventory)

↓ долг

Погашение → Доход партнёра

Расходы партнёра → Статистика админа

3. РОЛИ И ПРАВА ДОСТУПА

3.1 Администратор (Admin)

****Основные функции:****

- Управление каталогом товаров (CRUD)
- Управление системой расходов (Сюзерен/Вассал/Обыватель)
- Журнал производства (массовое добавление товаров)
- Одобрение/отклонение запросов партнёров (PartnerRequest)
- Управление областями и городами

- Активация/деактивация магазинов
- Просмотр глобальной статистики
- ****Мониторинг заказов магазинов**** (без одобрения)

****Специальные права:****

- Доступ ко всем магазинам системы
- Просмотр расходов всех партнёров
- Изменение цен товаров
- Удаление товаров из каталога
- Расчёт себестоимости и наценки

3.2 Партнёр (Partner)

****Основные функции:****

- Управление собственным инвентарём (PartnerInventory)
- Запросы товаров у админа (PartnerRequest: request/return)
- ****Приём предзаказов от магазинов**** (PENDING → IN_TRANSIT)
- ****Создание ручных заказов магазинам**** (manual orders)
- Отметка товаров для возврата (ReturnedItem)
- Указание суммы предоплаты
- Подтверждение заказов (IN_TRANSIT → ACCEPTED)
- Учёт собственных расходов (PartnerExpense)
- Приём погашений долгов от магазинов
- Отметка бракованных товаров (DefectiveProduct)

****Специальные права:****

- Удаление товаров из корзины магазина (при обработке предзаказа)
- Просмотр инвентаря магазинов
- Доступ к расширенной статистике

3.3 Магазин (Store)

****Основные функции:****

- Создание предзаказов (выбор из каталога админа)
- Просмотр собственного инвентаря (StoreInventory)
- Погашение долгов
- Просмотр истории заказов

- Заявка о бракованных товарах
- Редактирование данных магазина

****Ограничения:****

- Видит только свои магазины (`owner=request.user`)
- Не может удалять товары из инвентаря
- Не может изменять цены
- Не может создавать заказы при деактивации

4. АУТЕНТИФИКАЦИЯ

4.1 Скрытая валидация роли через маркер

****Механизм:****

1. При регистрации пользователь вводит пароль
2. Система проверяет наличие маркера в пароле
3. Если маркер найден → `role="partner"`
4. Если маркер не найден → `role="store"`
5. Маркер удаляется из пароля перед сохранением

****Требования к маркеру:****

- Длина: 3-5 символов
- Уникальный и неочевидный (пример: `p!8Rt`)
- Хранится в переменной окружения `PARTNER_MARKER`
- Вставляется в произвольной позиции пароля
- НЕ сохраняется в базе данных

****Пример:****

```
password = "12345p!8Rt67890"
```

```
marker = "p!8Rt"
```

```
→ role = "partner"
```

→ saved_password = "1234567890" (без маркера)

4.2 Функция "Запомнить меня"

****Стандартный вход (remember_me=false):****

- Access Token: 60 минут
- Refresh Token: 7 дней

****Долгосрочный вход (remember_me=true):****

- Access Token: 90 дней
- Refresh Token: 180 дней

4.3 Сброс пароля (3 этапа)

****Этап 1: Запрос кода****

POST /api/auth/password/reset/

Body: { "email": "user@example.com" }

Система:

1. Проверяет существование email
2. Генерирует 5-значный код (Integer)
3. Отправляет код на email
4. Код действителен 15 минут

****Этап 2: Верификация кода****

POST /api/auth/password/verify/

Body: { "email": "user@example.com", "code": "12345" }

Система:

1. Проверяет корректность кода
2. Проверяет срок действия
3. Возвращает токен для смены пароля

****Этап 3: Установка нового пароля****

POST /api/auth/password/confirm/

Body: { "token": "...", "new_password": "newpass123" }

Система:

1. Проверяет токен
2. Устанавливает новый пароль
3. Инвалидирует все старые сессии

4.4 Email валидация

****Правила:****

- Уникальный в системе
- Один символ @ (не более)
- Длина ≤ 50 символов
- Формат: username@domain.com

****Regex:****

```
^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$
```

4.5 Phone валидация

****Правила:****

- Уникальный в системе
- Формат: +996XXXXXXXXXX (13 символов)
- Только цифры после +996

****Regex:****

```
^\+996[0-9]{9}$
```

5. СИСТЕМА МАГАЗИНОВ

5.1 Owner-based доступ (КРИТИЧЕСКОЕ ИЗМЕНЕНИЕ v3.0)

****Принцип:****

```
# Каждый магазин принадлежит создателю

store.owner = request.user

# Пользователь видит только свои магазины

Store.objects.filter(owner=request.user)
```

****Правила:****

- Один пользователь может создать ****несколько магазинов****
- Один пользователь может быть ****только в одном магазине одновременно****
- Несколько пользователей ****НЕ МОГУТ**** быть в одном магазине (изменение от v2.0!)
- Пользователь может выйти и переключиться на другой свой магазин

5.2 Регистрация магазина

****Обязательные поля:****

- Название магазина
- Владелец магазина (ФИО)
- Номер телефона
- ИНН (12-14 символов, уникальный)
- Область (выбор из списка)
- Город (выбор из списка, зависит от области)
- Адрес магазина

****Валидация:****

- ИНН: только цифры, 12-14 символов
- Телефон: +996XXXXXXXXX
- Email (опционально): ≤ 50 символов, один @

****После регистрации:****

- Магазин создаётся со статусом `is_active=True`
- Ожидает одобрения админа (если требуется)
- Пользователь может входить в магазин

5.3 Статусы магазина

****Активный (`**is_active=True**`):****

- Может создавать заказы
- Может погашать долги
- Полный функционал

****Деактивированный (`**is_active=False**`):****

- ****НЕ может**** создавать новые заказы
- ****МОЖЕТ**** входить в систему
- ****НЕ может**** погашать долги (требуется уточнения)
- Отображается в списке, но с ограничениями

5.4 Фильтрация магазинов

****Админ и Партнёр:****

- По областям и городам
- По долгу (> 0, сортировка по убыванию)

- По погашенным (= 0, сортировка по дате погашения)
 - Поиск по: ИНН, название, город
-

6. КАТАЛОГ ТОВАРОВ

6.1 Типы товаров

****Штучные товары:****

- Единица измерения: штуки (шт)
- Могут быть бонусными
- Система бонусов: каждый 21-й бесплатно
- Примеры: пельмени, хинкали, мороженое

****Весовые товары:****

- Единица измерения: килограммы (кг)
- Минимум: 1 кг (или 0.1 кг если остаток < 1 кг)
- Шаг: 0.1 кг (100 грамм)
- ****НЕ могут быть бонусными****
- Могут быть бракованными
- Примеры: курица, мука, фасоль

6.2 Расчёт цены весовых товаров

****Формула:****

Цена за 100г = Цена за 1кг ÷ 10

Итоговая цена = (Цена за 100г) × (Вес ÷ 0.1)

****Примеры:****

Курица: 450 сом/кг, 2.5 кг

→ $(450 \div 10) \times (2.5 \div 0.1) = 45 \times 25 = 1,125$ сом

Мука: 50 сом/кг, 10 кг

→ $(50 \div 10) \times (10 \div 0.1) = 5 \times 100 = 500$ сом

6.3 Структура товара

****Модель Product:****

```
class Product(models.Model):  
  
    name = CharField(unique=True)           # Название  
  
    price = DecimalField()                  # Цена  
  
    quantity = DecimalField()               # Количество  
  
    is_weight_based = BooleanField()         # Весовой?  
  
    is_bonus = BooleanField()                # Бонусный?  
  
    description = TextField(max_length=250) # Описание  
  
    images = [...] (1-3 изображения)       # Картинки  
  
    is_active = BooleanField(default=True)   # Активен?
```

****Валидация:****

- ☒ Весовые товары ****НЕ могут**** быть бонусными
- ☒ Нельзя изменить тип с "весовой" на "штучный"
- ☒ Можно изменить цену в любой момент
- ☒ Изменение цены НЕ влияет на старые заказы

6.4 CRUD операции (только Админ)

****Создание товара:****

- Название (принимает все символы)
- Цена (только цифры)
- Количество (только цифры, без лимита)

- Описание (до 250 символов)
- Выбор: бонусный/обычный
- Изображения (1-3 шт, JPG/PNG, до 5 МБ)
- Нельзя создать товар с существующим названием

****Редактирование:****

- Можно изменить: цену, название, изображения, статус бонусный/обычный
- ****НЕЛЬЗЯ**** изменить тип "весовой" → "штучный"

****Удаление:****

- Подтверждение через модальное окно "Да/Нет"
- Товар полностью пропадает из каталога
- Партнёры больше не могут его продавать
- Проданные товары остаются в истории
- Остатки на складе исчезают

6.5 Кнопка "Отправить на склад"

****Функция:****

- Админ увеличивает количество товара в каталоге
- Используется для добавления товаров вручную
- Для массового производства используется Журнал производства

7. ПАРТНЁРСКИЙ ИНВЕНТАРЬ

7.1 Модель PartnerInventory

```
class PartnerInventory(models.Model):  
  
    partner = ForeignKey(User)                # Партнёр  
  
    product = ForeignKey(Product)             # Товар  
  
    quantity = DecimalField()                 # Количество  
  
    weight = DecimalField(null=True)          # Вес (для весовых)
```

```
partner_request_id = ForeignKey()
```

```
# Связь с запросом
```

7.2 Запросы товаров (PartnerRequest)

****Типы запросов:****

- REQUEST - запрос товаров у админа
- RETURN - возврат товаров админу

****Статусы:****

- PENDING - ожидает одобрения админа
- APPROVED - одобрено
- REJECTED - отклонено

****Workflow запроса товаров:****

1. Партнёр создаёт запрос (REQUEST) → PENDING
2. Админ просматривает в разделе "Запросы"
3. Админ одобряет → APPROVED
 - Товары списываются из каталога админа
 - Товары добавляются в PartnerInventory
4. Админ отклоняет → REJECTED
 - Указывает причину отказа

****Workflow возврата товаров:****

1. Партнёр создаёт запрос (RETURN) → PENDING
 - Товары резервируются в PartnerInventory
2. Админ одобряет → APPROVED
 - Товары удаляются из PartnerInventory

- Товары возвращаются в каталог админа

3. Админ отклоняет → REJECTED

- Резервирование снимается

8. WORKFLOW ЗАКАЗОВ (КРИТИЧЕСКОЕ ИЗМЕНЕНИЕ v3.0)

8.1 Предзаказ (Preorder - инициатива магазина)

****Полный цикл:****



| 3. Партнёр подтверждает

↓

┌──────────┐

| ACCEPTED | Товары → StoreInventory

└──────────┘

Долг создаётся

****Детально:****

****Шаг 1: Магазин создаёт заказ****

```
POST /api/stores/orders/
```

```
Body: {
```

```
  "items": [
```

```
    {"product": 5, "quantity": 10},      # Штучный
```

```
    {"product": 8, "quantity": 1, "weight": "2.5"} # Весовой
```

```
  ]
```

```
}
```

Результат:

```
- status = PENDING
```

```
- order_type = 'preorder'
```

```
- Товары в StoreOrderItem (НЕ в StoreInventory!)
```

****Шаг 2: Партнёр принимает заказ****

POST /api/partners/preorders/{id}/accept/

Результат:

- status = IN_TRANSIT
- Партнёр видит товары в "корзине"
- Может отмечать товары для возврата

****Шаг 3: Партнёр подтверждает заказ****

POST /api/partners/preorders/{id}/confirm/

Body: {

```
"returned_items": [  
    {"product": 5, "quantity": 5, "reason": "Нет в наличии"}  
],  
"prepayment": "500.00"  
}
```

Процесс:

1. Возвращённые товары → ReturnedItem
2. Оставшиеся товары: PartnerInventory → StoreInventory
3. Расчёт бонусов (если штучные)
4. Долг = total_amount - prepayment - returned_items

```
5. status = ACCEPTED
```

8.2 Ручной сбор (Manual - инициатива партнёра)

****Полный цикл:****



****Детально:****

```
POST /api/partners/manual-orders/
```

```
Body: {
```

```
  "store": 1,
```

```
  "items": [
```

```
    {"product": 5, "quantity": 20}
```

```
],  
  
  "prepayment": "1000.00",  
  
  "returned_items": [] # опционально  
}
```

Процесс:

1. Проверка наличия товаров в PartnerInventory
2. Списание: PartnerInventory → StoreInventory
3. Долг = total_amount - prepayment - returned_items
4. status = ACCEPTED
5. order_type = 'manual'

8.3 Роль админа в заказах

****Админ НЕ одобряет заказы магазинов!****

Админ видит заказы в разделе *****Запросы → Магазины***** только для:

- Мониторинга
- Статистики
- Анализа

9. СИСТЕМА ДОЛГОВ

9.1 Создание долга

****Момент создания:****

- При подтверждении партнёром (ACCEPTED)

****Формула:****

```
долг = total_amount - prepayment - returned_items_amount
```

****Пример:****

Заказ: 10 шт Пельменей (200 сом) = 2,000 сом

2.5 кг Курицы (450 сом/кг) = 1,125 сом

Итого: 3,125 сом

Партнёр возвращает: 5 шт Пельменей = 1,000 сом

Предоплата: 500 сом

Долг = 3,125 - 1,000 - 500 = 1,625 сом

9.2 Погашение долга

****Модель DebtPayment:****

```
class DebtPayment(models.Model):  
  
    order = ForeignKey(StoreOrder)  
  
    amount = DecimalField()  
  
    paid_by = ForeignKey(User)           # Магазин  
  
    received_by = ForeignKey(User)       # Партнёр/Админ  
  
    comment = TextField()  
  
    created_at = DateTimeField()
```


****Процесс:****

```
POST /api/stores/orders/{id}/pay-debt/
```

```
Body: {
```

```
    "amount": "500.00",
```

```
    "comment": "Частичное погашение"
```

```
}
```

Результат:

- Store.paid_debt += amount
- Store.total_debt -= amount
- Погашенный долг идёт в доход партнёра

****Типы погашения:****

- Частичное: amount < долг
- Полное: amount = долг
- Переплата: amount > долг (долг становится отрицательным)

10. БОНУСНАЯ СИСТЕМА

10.1 Правило "Каждый 21-й бесплатно"

****Механизм:****

20 платных товаров + 1 бонусный = 21 товар

bonus_count = total_quantity ÷ 21 # целочисленное деление

```
paid_count = total_quantity - bonus_count
```

****Примеры:****

50 шт → бонусов: 2, платных: 48

100 шт → бонусов: 4, платных: 96

20 шт → бонусов: 0, платных: 20

21 шт → бонусов: 1, платных: 20

10.2 Кумулятивный характер

****Бонусы считаются по инвентарю магазина:****

Заказ #1: 15 шт → Инвентарь: 15 (бонус: 0)

Заказ #2: 10 шт → Инвентарь: 25 (бонус: 1)

Заказ #3: 20 шт → Инвентарь: 45 (бонус: 2)

10.3 Ограничения

****Бонусные товары:****

-  Только штучные товары
-  Весовые товары НЕ могут быть бонусными
-  Автоматический расчёт при добавлении в инвентарь
-  Админ может вручную сделать товар бонусным (игнорирует правило 21)

****Влияние на статистику:****

- Бонусы НЕ влияют на сумму дохода
 - Бонусы считаются только количественно
 - Отображаются отдельно в истории
-

11. ВОЗВРАЩЁННЫЕ ТОВАРЫ

11.1 Модель ReturnedItem

```
class ReturnedItem(models.Model):  
  
    order = ForeignKey(StoreOrder)  
  
    product = ForeignKey(Product)  
  
    quantity = DecimalField()  
  
    weight = DecimalField(null=True)  
  
    price = DecimalField()  
  
    reason = TextField(null=True)  
  
    returned_at = DateTimeField()  
  
    returned_by = ForeignKey(User)      # Партнёр
```

11.2 Когда создаются

****Предзаказ:****

- Партнёр отмечает товары для возврата при обработке заказа (IN_TRANSIT)
- Товары НЕ попадают в StoreInventory
- Уменьшают сумму заказа

****Ручной сбор:****

- Партнёр может отметить товары для возврата при создании заказа
- Принцип тот же

11.3 Отображение

****В истории магазина:****

Возвращённые товары:

- Пельмени красные 10 шт 1,000 сом
- Курица сырая 2 кг 900 сом

****В статистике:****

- Не влияют на доход
- Не влияют на долг (уже вычтены из суммы заказа)
- Отображаются отдельным полем

12. БРАКОВАННЫЕ ТОВАРЫ

12.1 Модель DefectiveProduct

```
class DefectiveProduct(models.Model):  
  
    store = ForeignKey(Store)  
  
    product = ForeignKey(Product)  
  
    order = ForeignKey(StoreOrder, null=True)  
  
    quantity = DecimalField()  
  
    weight = DecimalField(null=True)  
  
    defect_amount = DecimalField()  
  
    notes = TextField()  
  
    reported_at = DateTimeField()  
  
    confirmed_by = ForeignKey(User)      # Партнёр
```

12.2 Workflow

1. Магазин заявляет о браке

2. Партнёр проверяет заявку
3. Партнёр подтверждает → брак списывается
 - Уменьшение долга: `Store.total_debt -= defect_amount`
 - Списание из инвентаря: `StoreInventory.defective_count++`
 - Обновление статистики: `defective_amount`
4. Партнёр отклоняет → заявка удаляется

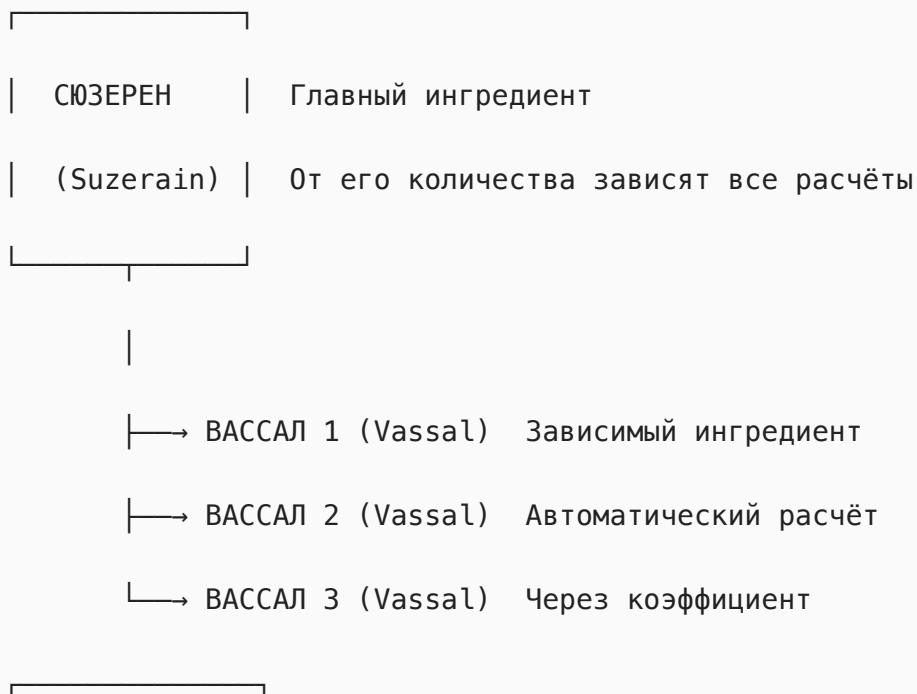
12.3 Весовые товары

****Для весовых:****

- Фиксируется вес брака (например, 1.3 кг)
- Рассчитывается сумма брака
- Пример: 1.3 кг курицы × 450 сом/кг = 585 сом

13. СИСТЕМА РАСХОДОВ (Сюзерен/Вассал/Обыватель)

13.1 Иерархия расходов



ОБЫВАТЕЛЬ	Независимый расход
(Civilian)	Может быть универсальным или обычным

13.2 Типы расходов

****Сюзерен (Suzerain):****

- Главный ингредиент производства
- Пример: Фарш ($35 \text{ кг} \times 810 \text{ сом/кг} = 28,350 \text{ сом}$)
- Статус устанавливается автоматически при выборе "механический учёт"

****Вассал (Vassal):****

- Зависит от Сюзерена через коэффициент
- Автоматически рассчитывается
- Состояние: механическое
- Пример: Мука ($35 \text{ кг} \times 0.095 = 3.325 \text{ кг} \times 40 \text{ сом/кг} = 133 \text{ сом}$)

****Обыватель (Civilian):****

- Независимый расход
- Может быть универсальным (на все товары) или обычным (на выбранные)
- Примеры: Аренда, Зарплата, Электроэнергия

13.3 Состояния расходов

****Механическое:****

- Партнёр вручную вводит потраченную сумму
- Статус становится "Вассал"
- Редактируется на главном экране

****Автоматическое:****

- Рассчитывается от механических расходов
- Не редактируется напрямую

13.4 Типы применения

****Обычный:****

- Применяется к выбранным товарам вручную
- Партнёр выбирает товары и указывает пропорции

****Универсальный:****

- Применяется ко всем товарам автоматически
- Распределяется пропорционально объёму производства

14. ЖУРНАЛ ПРОИЗВОДСТВА (НОВОЕ v3.0)

14.1 Назначение

****Для массового производства:****

- Админ производит 50,000 - 200,000 товаров за раз
- Считает расходы "мешками" (например, мешок приправы на 200,000 шт)
- Рассчитывает себестоимость партии

14.2 Workflow

1. Админ открывает "Журнал производства"
2. Выбирает товар (например, Пельмени)
3. Вводит количество произведённых единиц (200,000 шт)
4. Вводит расходы партии:
 - Физические (ингредиенты)
 - Накладные (аренда, зарплата за период)
5. Система рассчитывает себестоимость за 1 шт
6. Админ сохраняет партию
7. Товары добавляются в каталог (+200,000 шт)

14.3 Расчёт себестоимости

****Формула:****

Дневные расходы

$\text{daily_expenses} = \text{физические_расходы} + \text{накладные_расходы}$

Месячные расходы (пересчёт в день)

$\text{monthly_per_day} = \text{месячные_расходы} \div 30$

Общие расходы за период производства

$\text{total_expenses} = \text{daily_expenses} + \text{monthly_per_day}$

Себестоимость за единицу

$\text{cost_per_unit} = \text{total_expenses} \div \text{quantity_produced}$

****Пример:****

Произведено: 1,100 шт/день

Дневные расходы: 80,740 сом

Месячные расходы: 189,000 сом

Месячные (в день): $189,000 \div 30 = 6,300$ сом

Общие: $80,740 + 6,300 = 87,040$ сом/день

Себестоимость: $87,040 \div 1,100 = 79.13$ сом/шт

15. СЕБЕСТОИМОСТЬ И НАЦЕНКА

15.1 Расчёт себестоимости

****Компоненты:****

- Физические расходы (ингредиенты)
- Накладные расходы (аренда, зарплата, и т.д.)
- Пропорциональное распределение

****Система отображения:****

- Количество штук в упаковке
- Себестоимость за 1 шт
- Цена продажи
- Чистая прибыль
- Наценка (%)

15.2 Установка цены

****Админ может:****

- Установить наценку в процентах
- Или указать сумму наценки вручную

****Автоматический расчёт:****

```
# Вариант 1: Наценка в процентах

markup_percent = 400 # %

selling_price = cost_per_unit × (1 + markup_percent/100)

# 79.13 × 5 = 395.65 сом

# Вариант 2: Наценка вручную

markup_amount = 170.87 сом

selling_price = cost_per_unit + markup_amount
```

79.13 + 170.87 = 250 сом

****Отображение:****

Себестоимость за 1 шт:	30.3 Сом
Цена продажи:	245 Сом/кг
Чистая прибыль:	1061 сом/кг
Наценка:	+400%

16. СТАТИСТИКА

16.1 Статистика Админа

****Круговая диаграмма (4 сегмента):****

- Доход (синий)
- Расход (фиолетовый)
- Долг (красный)
- Бонус (бирюзовый)

****Поля:****

Доход:	297,000 с	
Зеленые пельмени:	5234 шт	Все товары
Красные пельмени:	3734 шт	275,000 с
Пельмени Тойбос:	2344 шт	245,000 с

Расходы: 145,000 с

Долг: 10,000 с

Погашенный долг: 45,000 с

Бонусы: 3,000 с

Брак: 230 шт

Остаток: 6078 шт

Общий баланс: 602,000 С

****Фильтры:****

- Поиск магазинов (по ИНН, названию, городу)
- Календарь (день, неделя, месяц, полгода, год, за всё время)
- Область и город

16.2 Статистика Партнёра

****Круговая диаграмма (3 сегмента):****

- Доход (синий)
- Долг (красный)
- Бонус (бирюзовый)

****Поля (аналогично админу, но БЕЗ расходов в круге):****

Доход: 275,000 с

Долг:	10,000 с
Погашенный долг:	45,000 с
Бонусы:	3,000 с
Брак:	230 шт
Общий баланс:	602,000 С

****Расходы партнёра:****

- Отображаются отдельно в поле
- НЕ отображаются в круговой диаграмме
- Влияют на статистику админа

17. ГЕОГРАФИЯ (Области и города)

17.1 Модели

```
class Region(models.Model):  
  
    name = CharField(unique=True)  
  
class City(models.Model):  
  
    name = CharField()  
  
    region = ForeignKey(Region)
```

17.2 CRUD операции (только Админ)

****Создание области:****

```
POST /api/admin/regions/
```

```
Body: { "name": "Нарынская область" }
```

****Создание города:****

```
POST /api/admin/cities/
```

```
Body: {  
    "name": "Нарын",  
    "region": 1  
}
```

****Удаление:****

- При удалении области/города они пропадают из выбора магазинов
- Существующие магазины сохраняют привязку
- Можно выбрать заново

17.3 Фильтрация

****В списке городов:****

- Фильтр по областям
- "Все области" - показать все города
- Поиск по названию

18. ЧАТЫ И УВЕДОМЛЕНИЯ

18.1 Чаты (Django Channels + WebSocket)

****Типы чатов:****

- Админ ↔ Партнёр
- Партнёр ↔ Магазин
- Админ ↔ Магазин

****Функционал:****

- Отправка текста, изображений, видео, документов, аудио
- Удаление сообщений (у всех / только у себя)
- Группировка сообщений по датам
- История с пагинацией
- JWT-авторизация для WebSocket

****Форматы файлов:****

- Фото: JPEG, PNG
- Документы: PDF
- Видео: MP4
- Аудио: MP3
- Лимит: до 10-20 МБ

18.2 Уведомления (FCM + WebSocket)

****Типы уведомлений:****

1. Новое сообщение из чатов
2. Изменился статус заказа
3. Сделан новый заказ
4. Добавился новый магазин
5. Сделан расход (для партнёров)
6. Пришёл заказ (для партнёров)

****Модель Notification:****

```
class Notification(models.Model):  
  
    user = ForeignKey(User)  
  
    notification_type = CharField()  
  
    title = CharField()  
  
    message = TextField()  
  
    related_order_id = ForeignKey(StoreOrder, null=True)  
  
    related_store_id = ForeignKey(Store, null=True)  
  
    is_read = BooleanField(default=False)
```

```
created_at = DateTimeField()
```

****Функционал:****

- Календарная фильтрация
 - Отметить все прочитанными
 - Удалить все
 - Звук и вибрация (одинаковые для всех типов)
-

19. API ENDPOINTS

19.1 Authentication

```
POST    /api/auth/register/
POST    /api/auth/login/
POST    /api/auth/logout/
POST    /api/auth/refresh/
POST    /api/auth/password/reset/
POST    /api/auth/password/verify/
POST    /api/auth/password/confirm/
```

19.2 Products (Admin)

GET	/api/products/	# Список товаров
POST	/api/products/	# Создать товар
GET	/api/products/{id}/	# Детали товара
PUT	/api/products/{id}/	# Обновить товар

DELETE	/api/products/{id}/	# Удалить товар
POST	/api/products/{id}/add-stock/	# Добавить на склад

19.3 Partner Inventory

GET	/api/partners/inventory/	# Инвентарь партнёра
POST	/api/partners/requests/	# Создать запрос
GET	/api/partners/requests/	# Список запросов
GET	/api/partners/requests/{id}/	# Детали запроса

19.4 Partner Requests (Admin)

GET	/api/admin/partner-requests/	# Список запросов
GET	/api/admin/partner-requests/{id}/	# Детали
POST	/api/admin/partner-requests/{id}/approve/	# Одобрить
POST	/api/admin/partner-requests/{id}/reject/	# Отклонить
DELETE	/api/admin/partner-requests/{id}/	# Удалить

19.5 Store Orders

# Магазин		
POST	/api/stores/orders/	# Создать предзаказ
GET	/api/stores/orders/	# Мои заказы
GET	/api/stores/orders/{id}/	# Детали заказа

Партнёр – Предзаказы

GET	/api/partners/preorders/	# Список предзаказов
POST	/api/partners/preorders/{id}/accept/	# Принять (→ IN_TRANSIT)
POST	/api/partners/preorders/{id}/confirm/	# Подтвердить (→ ACCEPTED)
POST	/api/partners/preorders/{id}/reject/	# Отклонить

Партнёр – Ручной сбор

POST	/api/partners/manual-orders/	# Создать ручной заказ
------	------------------------------	------------------------

Админ – Мониторинг

GET	/api/admin/store-orders/	# Все заказы магазинов
GET	/api/admin/store-orders/{id}/	# Детали

19.6 Stores

GET	/api/stores/	# Список магазинов
POST	/api/stores/	# Создать магазин
GET	/api/stores/{id}/	# Детали магазина
PUT	/api/stores/{id}/	# Обновить
DELETE	/api/stores/{id}/	# Удалить
POST	/api/stores/{id}/activate/	# Активировать
POST	/api/stores/{id}/deactivate/	# Деактивировать
GET	/api/stores/{id}/history/	# История магазина

POST	/api/stores/{id}/pay-debt/	# Погасить долг
------	----------------------------	-----------------

19.7 Defective Products

POST	/api/stores/{id}/report-defective/	# Заявить о браке (магазин)
GET	/api/partners/defective-products/	# Список заявок (партнёр)
POST	/api/partners/defective/{id}/confirm/	# Подтвердить брак
POST	/api/partners/defective/{id}/reject/	# Отклонить заявку

19.8 Expenses

GET	/api/expenses/	# Список расходов (Admin)
POST	/api/expenses/	# Создать расход (Admin)
PUT	/api/expenses/{id}/	# Обновить расход
DELETE	/api/expenses/{id}/	# Удалить расход
POST	/api/partners/expenses/	# Добавить расход партнёра
GET	/api/partners/expenses/	# Список расходов партнёра

19.9 Production Journal

GET	/api/admin/production-journal/	# Список партий
POST	/api/admin/production-journal/	# Добавить партию
GET	/api/admin/production-journal/{id}/	# Детали партии

19.10 Reports

GET	/api/reports/admin-stats/	# Статистика админа
GET	/api/reports/partner-stats/	# Статистика партнёра
GET	/api/reports/store-history/	# История магазина

19.11 Geography

GET	/api/regions/	# Список областей
POST	/api/admin/regions/	# Создать область
PUT	/api/admin/regions/{id}/	# Обновить область
DELETE	/api/admin/regions/{id}/	# Удалить область
GET	/api/cities/	# Список городов
POST	/api/admin/cities/	# Создать город
PUT	/api/admin/cities/{id}/	# Обновить город
DELETE	/api/admin/cities/{id}/	# Удалить город

19.12 Messaging

GET	/api/chats/	# Список чатов
GET	/api/chats/{id}/messages/	# История сообщений
POST	/api/chats/{id}/messages/	# Отправить сообщение
DELETE	/api/messages/{id}/	# Удалить сообщение

WebSocket: /ws/chat/{room_id}/

19.13 Notifications

GET	/api/notifications/	# Список уведомлений
POST	/api/notifications/mark-read/	# Отметить прочитанными
DELETE	/api/notifications/clear-all/	# Удалить все
POST	/api/fcm-tokens/	# Зарегистрировать FCM токен

20. МОДЕЛИ ДАННЫХ

20.1 User

```
class User(AbstractUser):  
  
    phone = CharField(unique=True) # USERNAME_FIELD  
  
    email = EmailField(unique=True)  
  
    name = CharField()  
  
    second_name = CharField()  
  
    role = CharField(choices=['admin', 'partner', 'store'])  
  
    approval_status = CharField()  
  
    is_active = BooleanField(default=True)  
  
    created_at = DateTimeField(auto_now_add=True)
```

20.2 Product

```
class Product(models.Model):  
  
    name = CharField(unique=True)  
  
    price = DecimalField()  
  
    quantity = DecimalField()  
  
    is_weight_based = BooleanField()  
  
    is_bonus = BooleanField()  
  
    description = TextField(max_length=250)  
  
    is_active = BooleanField(default=True)
```

20.3 Store

```
class Store(models.Model):  
  
    name = CharField(unique=True)  
  
    owner = ForeignKey(User) # v3.0: owner-based!  
  
    owner_name = CharField()  
  
    phone = CharField()  
  
    inn = CharField(unique=True) # 12-14 СИМВОЛОВ  
  
    region = ForeignKey(Region)  
  
    city = ForeignKey(City)  
  
    address = TextField()  
  
    is_active = BooleanField(default=True)  
  
    total_debt = DecimalField(default=0)
```

```
paid_debt = DecimalField(default=0)
```

20.4 PartnerInventory

```
class PartnerInventory(models.Model):  
  
    partner = ForeignKey(User)  
  
    product = ForeignKey(Product)  
  
    quantity = DecimalField()  
  
    weight = DecimalField(null=True)  
  
    partner_request_id = ForeignKey(PartnerRequest)
```

20.5 PartnerRequest

```
class PartnerRequest(models.Model):  
  
    partner = ForeignKey(User)  
  
    product = ForeignKey(Product)  
  
    request_type = CharField(choices=['request', 'return'])  
  
    status = CharField(choices=['pending', 'approved', 'rejected'])  
  
    quantity = DecimalField()  
  
    reason = TextField()  
  
    reviewed_by = ForeignKey(User, null=True)  
  
    reviewed_at = DateTimeField(null=True)  
  
    rejection_reason = TextField()  
  
    created_at = DateTimeField(auto_now_add=True)
```

20.6 StoreOrder

```
class StoreOrder(models.Model):

    store = ForeignKey(Store)

    status = CharField(choices=['pending', 'in_transit', 'accepted',
                              'rejected'])

    order_type = CharField(choices=['preorder', 'manual'])

    total_amount = DecimalField()

    prepayment = DecimalField(default=0)

    debt_amount = DecimalField(default=0)

    bonus_count = IntegerField(default=0)

    defective_count = IntegerField(default=0)

    notes = TextField()

    rejection_reason = TextField()

    created_at = DateTimeField()

    accepted_at = DateTimeField(null=True)

    confirmed_by = ForeignKey(User, null=True) # Партнёр
```

20.7 StoreOrderItem

```
class StoreOrderItem(models.Model):

    order = ForeignKey(StoreOrder)

    product = ForeignKey(Product)

    quantity = DecimalField()

    weight = DecimalField(null=True)
```

```
price_at_order = DecimalField()

total_price = DecimalField()

is_bonus = BooleanField(default=False)
```

20.8 StoreInventory

```
class StoreInventory(models.Model):

    store = ForeignKey(Store)

    product = ForeignKey(Product)

    quantity = DecimalField()

    weight = DecimalField(null=True)

    bonus_count = IntegerField(default=0)

    paid_count = IntegerField(default=0)

    defective_count = IntegerField(default=0)
```

20.9 ReturnedItem

```
class ReturnedItem(models.Model):

    order = ForeignKey(StoreOrder)

    product = ForeignKey(Product)

    quantity = DecimalField()

    weight = DecimalField(null=True)

    price = DecimalField()

    reason = TextField(null=True)
```



```
returned_at = DateTimeField()

returned_by = ForeignKey(User)           # Партнёр
```

20.10 DefectiveProduct

```
class DefectiveProduct(models.Model):

    store = ForeignKey(Store)

    product = ForeignKey(Product)

    order = ForeignKey(StoreOrder, null=True)

    quantity = DecimalField()

    weight = DecimalField(null=True)

    defect_amount = DecimalField()

    notes = TextField()

    reported_at = DateTimeField()

    confirmed_by = ForeignKey(User, null=True) # Партнёр
```

20.11 DebtPayment

```
class DebtPayment(models.Model):

    order = ForeignKey(StoreOrder)

    amount = DecimalField()

    paid_by = ForeignKey(User)           # Магазин

    received_by = ForeignKey(User)       # Партнёр/Админ

    comment = TextField()
```

```
created_at = DateTimeField()
```

20.12 Expense

```
class Expense(models.Model):

    name = CharField()

    expense_type = CharField(choices=['physical', 'overhead'])

    expense_status = CharField(choices=['suzerain', 'vassal', 'civilian'])

    expense_state = CharField(choices=['mechanical', 'automatic'])

    apply_type = CharField(choices=['regular', 'universal'])

    unit_type = CharField(choices=['per_piece', 'per_weight'])

    monthly_amount = DecimalField()

    daily_amount = DecimalField()

    quantity = DecimalField()

    unit_cost = DecimalField()

    depends_on_suzerain = ForeignKey('self', null=True)

    dependency_ratio = DecimalField(null=True)
```

20.13 PartnerExpense

```
class PartnerExpense(models.Model):

    partner = ForeignKey(User)

    amount = DecimalField()

    description = TextField()
```

```
expense_date = DateField()

created_at = DateTimeField()
```

ЗАКЛЮЧЕНИЕ

Это ****ФИНАЛЬНОЕ КОНСОЛИДИРОВАННОЕ ТЕХНИЧЕСКОЕ ЗАДАНИЕ БайЭл v3.0**** без противоречий.

****Основано на:****

-  Анализ всех технических заданий
-  Изучение текущего кода v2.0
-  Детальный анализ дизайна Figma (все 3 роли)
-  Уточнения от клиента по всем критическим вопросам

****Критические изменения v3.0:****

1. Owner-based система магазинов
2. Заказы магазина напрямую к партнёру (минуя админа)
3. Партнёрский инвентарь
4. Два типа заказов (предзаказ + ручной сбор)
5. Возвращённые товары
6. Журнал производства
7. Расширенная система расходов

****Документ готов к разработке!**** 

13. СИСТЕМА РАСХОДОВ (Сюзерен/Вассал/Обыватель)

13.1 Иерархия расходов

****Структура:****

СЮЗЕРЕН (Suzerain)

└─> Главный ингредиент

└─ От него зависят Вассалы

└─ Механический учёт

ВАССАЛ (Vassal)

└─ Зависимый ингредиент

└─ Коэффициент от Сюзерена

└─ Автоматический расчёт

ОБЫВАТЕЛЬ (Civilian)

└─ Независимый расход

└─ Универсальный или обычный

└─ Ручной ввод

13.2 Типы расходов

****Физические (Physical):****

- Ингредиенты производства
- Примеры: мука, фарш, лук, тесто

****Накладные (Overhead):****

- Аренда, налоги, зарплата
- Электроэнергия, вода, доставка

13.3 Пример из дизайна

****Сюзерен:****

Говядина: $6.2\text{кг} \times 250 \text{ Сом} = 1,550 \text{ Сом}$

****Вассалы:****

Мука: $1.2\text{кг} \times 100 \text{ Сом} = 120 \text{ Сом}$ (коэффициент 0.19 от Говядины)

Специи: автоматический расчёт

****Обыватели:****

Локальные затраты:

- Сырьё / Закупка: 400 Сом
- Логистика: 500 Сом
- Зарплата: 150 Сом

Налоги и комиссии:

- Налог (5%): 35 сом
- Комиссия (3%): 21 сом

13.4 Расчёт себестоимости

****Формула:****

$$\text{total_expenses} = \Sigma(\text{Suzerains} + \text{Vassals}) + \Sigma(\text{Civilians})$$
$$\text{cost_per_unit} = \text{total_expenses} \div \text{quantity_produced}$$

****Пример:****

Произведено: 1,100 шт/день

Физические расходы: 80,740 сом

Накладные расходы: 6,300 сом/день (189,000/30)

Общие: 87,040 сом

Себестоимость: $87,040 \div 1,100 = 79.13$ сом/шт

14. ЖУРНАЛ ПРОИЗВОДСТВА

14.1 Назначение

****Для массового производства:****

- Админ производит 50,000 - 200,000 шт за раз
- Считает "мешками" ингредиентов
- Рассчитывает себестоимость партии

14.2 Модель ProductionBatch

```
class ProductionBatch(models.Model):  
  
    product = models.ForeignKey(Product)  
  
    quantity = models.DecimalField() # Количество произведённого  
  
    physical_expenses = models.DecimalField()  
  
    overhead_expenses = models.DecimalField()  
  
    cost_per_unit = models.DecimalField()  
  
    selling_price = models.DecimalField()  
  
    markup_percent = models.DecimalField()  
  
    created_at = models.DateTimeField()
```

```
notes = models.TextField()
```

14.3 Workflow

1. Админ открывает "Журнал производства"
2. Выбирает товар (Пельмени)
3. Вводит количество: 200,000 шт
4. Вводит расходы:
 - Физические: Фарш 35 кг × 810 = 28,350 сом
 - Накладные: Аренда 1000, Зарплата 500 = 1,500 сом
5. Система рассчитывает: $29,850 \div 200,000 = 0.149$ сом/шт
6. Админ устанавливает наценку: 400%
7. Цена продажи: $0.149 \times 5 = 0.745$ сом/шт → 745 сом/кг
8. Сохранение: `Product.quantity += 200,000`

14.4 Endpoint

```
POST /api/admin/production-journal/
```

****Body:****

```
{  
  
  "product": 5,  
  
  "quantity": 200000,  
  
  "physical_expenses": 28350.00,
```

```
"overhead_expenses": 1500.00,  
  
"markup_percent": 400,  
  
"notes": "Партия 30.12.2023"  
  
}
```

15. СЕБЕСТОИМОСТЬ И НАЦЕНКА

15.1 Экран "Цена и прибыль"

****Отображение:****

Пельмени (1 кг)

Количество штук в упаковке: 20 шт

Себестоимость за 1 шт: 30.3 Сом

Цена продажи: 245 Сом/ кг

Чистая прибыль: 1061 сом/ кг

Наценка: +400 %

15.2 Установка наценки

****Вариант 1: Процент****


```
markup_percent = 400 # %  
  
selling_price = cost_per_unit × (1 + markup_percent/100)  
  
# 79.13 × 5 = 395.65 сом/кг
```

****Вариант 2: Сумма****

```
markup_amount = 170.87 сом  
  
selling_price = cost_per_unit + markup_amount  
  
# 79.13 + 170.87 = 250 сом/кг
```

15.3 Endpoint

```
POST /api/admin/products/{id}/set-pricing/
```

****Body:****

```
{  
  "markup_percent": 400  
}  
  
// ИЛИ  
  
{  
  "markup_amount": 170.87  
}
```

}

16. СТАТИСТИКА

16.1 Админ - Статистика

****Круговая диаграмма (4 сегмента):****

-  Доход (синий)
-  Расход (фиолетовый)
-  Долг (красный)
-  Бонус (бирюзовый)

****Поля:****

Доход: 297,000 с

Зеленые пельмени 5234 шт	Все товары
--------------------------	------------

Красные пельмени 3734 шт	275,000 с
--------------------------	-----------

Пельмени Тойбос 2344 шт	245,000 с
-------------------------	-----------

Расходы: 145,000 с

Долг: 10,000 с

Погашенный долг: 45,000 с

Бонусы: 3,000 с

Брак: 230 шт

Остаток: 6078 шт

Общий баланс: 602,000 С

****Формула баланса:****

```
balance = income - expenses - defects - debt + paid_debt
```

16.2 Партнёр - Статистика

****Круговая диаграмма (3 сегмента):****

-  Доход (синий)
-  Долг (красный)
-  Бонус (бирюзовый)

****Отличие от админа:****

-  НЕТ расходов в круговой диаграмме
-  Расходы партнёра отображаются отдельным полем

16.3 Календарь

****Фильтры:****

- За неделю
- За месяц
- За полгода
- За год
- За всё время
- Кастомный период

16.4 Endpoint

```
GET /api/reports/admin-stats/?period=month&region=1&city=5
```

****Response:****

```
{
  "income": {
    "total": 297000,
    "by_product": [
      {
        "product": "Зеленые пельмени",
        "quantity": 5234,
        "amount": 120000
      }
    ]
  },
  "expenses": 145000,
  "debt": 10000,
  "paid_debt": 45000,
  "bonus": 3000,
  "defective": 230,
  "balance": 602000
}
```

17. ГЕОГРАФИЯ (Области и города)

17.1 Модели

```
class Region(models.Model):  
    name = models.CharField(max_length=100, unique=True)  
  
class City(models.Model):  
    name = models.CharField(max_length=100)  
    region = models.ForeignKey(Region, on_delete=models.CASCADE)  
  
class Meta:  
    unique_together = ['name', 'region']
```

17.2 CRUD (Admin only)

****Создание области:****

```
POST /api/admin/regions/  
  
Body: { "name": "Нарынская область" }
```

****Создание города:****

```
POST /api/admin/cities/  
  
Body: { "name": "Нарын", "region": 1 }
```

****Удаление:****

```
DELETE /api/admin/regions/{id}/
```

****Последствия удаления:****

- Область/город пропадает из выбора
- Существующие магазины сохраняют привязку
- Можно выбрать заново при редактировании

18. ЧАТЫ И УВЕДОМЛЕНИЯ

18.1 Чаты (WebSocket)

****Модели:****

```
class ChatRoom(models.Model):

    admin = models.ForeignKey(User, related_name='admin_chats')

    partner = models.ForeignKey(User, related_name='partner_chats',
                                null=True)

    store = models.ForeignKey(User, related_name='store_chats', null=True)

    created_at = models.DateTimeField(auto_now_add=True)
```

```
class Message(models.Model):

    room = models.ForeignKey(ChatRoom)

    sender = models.ForeignKey(User)

    message_type = models.CharField(choices=[

        ('text', 'Текст'),

        ('image', 'Изображение'),

        ('video', 'Видео'),

        ('document', 'Документ'),

        ('audio', 'Аудио')])
```

```
] )

content = models.TextField()

file_url = models.URLField(null=True)

is_deleted_for_all = models.BooleanField(default=False)

deleted_by_users = models.ManyToManyField(User)

created_at = models.DateTimeField(auto_now_add=True)
```

****WebSocket URL:****

```
ws://localhost:8000/ws/chat/{room_id}/?token=<JWT_TOKEN>
```

****Форматы файлов:****

- Фото: JPG, PNG (до 10 МБ)
- Видео: MP4 (до 20 МБ)
- Документы: PDF (до 10 МБ)
- Аудио: MP3 (до 10 МБ)

18.2 Уведомления

****Модель:****

```
class Notification(models.Model):

    user = models.ForeignKey(User)

    notification_type = models.CharField(choices=[

        ('new_message', 'Новое сообщение'),

        ('order_status_changed', 'Статус заказа изменён'),

        ('new_order', 'Новый заказ'),
```

```
        ('new_store', 'Новый магазин'),

        ('expense_added', 'Добавлен расход'),

        ('order_received', 'Заказ получен')

    ])

    title = models.CharField(max_length=255)

    message = models.TextField()

    related_order_id = models.IntegerField(null=True)

    related_store_id = models.IntegerField(null=True)

    is_read = models.BooleanField(default=False)

    created_at = models.DateTimeField(auto_now_add=True)
```

****Push через FCM:****

```
def send_push_notification(user, title, body, data={}):

    message = messaging.Message(

        notification=messaging.Notification(

            title=title,

            body=body,

        ),

        data=data,

        token=user.fcm_token

    )

    messaging.send(message)
```

19. API ENDPOINTS (ПОЛНЫЙ СПИСОК)

Authentication

```
POST    /api/auth/register/
POST    /api/auth/login/
POST    /api/auth/logout/
POST    /api/auth/refresh/
POST    /api/auth/password/reset/
POST    /api/auth/password/verify/
POST    /api/auth/password/confirm/
```

Products (Admin)

```
GET      /api/products/
POST     /api/admin/products/
GET      /api/products/{id}/
PUT      /api/admin/products/{id}/
DELETE   /api/admin/products/{id}/
POST     /api/admin/products/{id}/add-stock/
POST     /api/admin/products/{id}/set-pricing/
```

Partner Inventory

```
GET      /api/partners/inventory/
```

```
POST    /api/partners/requests/

GET     /api/partners/requests/

GET     /api/partners/requests/{id}/
```

Partner Requests (Admin)

```
GET     /api/admin/partner-requests/

GET     /api/admin/partner-requests/{id}/

POST    /api/admin/partner-requests/{id}/approve/

POST    /api/admin/partner-requests/{id}/reject/

DELETE  /api/admin/partner-requests/{id}/
```

Store Orders

Store

```
POST    /api/stores/orders/                # Создать предзаказ

GET     /api/stores/orders/                # Мои заказы

GET     /api/stores/orders/{id}/          # Детали

POST    /api/stores/orders/{id}/pay-debt/ # Погасить долг
```

Partner – Предзаказы

```
GET     /api/partners/preorders/

POST    /api/partners/preorders/{id}/accept/ # PENDING → IN_TRANSIT

POST    /api/partners/preorders/{id}/confirm/ # IN_TRANSIT → ACCEPTED
```

POST /api/partners/preorders/{id}/reject/

Partner – Ручной сбор

POST /api/partners/manual-orders/

Admin – Мониторинг

GET /api/admin/store-orders/

GET /api/admin/store-orders/{id}/

Stores

GET /api/stores/

POST /api/stores/

GET /api/stores/{id}/

PUT /api/stores/{id}/

DELETE /api/stores/{id}/

POST /api/admin/stores/{id}/activate/

POST /api/admin/stores/{id}/deactivate/

GET /api/stores/{id}/history/

POST /api/stores/{id}/report-defective/

Defective Products

GET /api/partners/defective-products/

POST /api/partners/defective/{id}/confirm/

POST /api/partners/defective/{id}/reject/

Expenses

GET /api/expenses/

POST /api/admin/expenses/

PUT /api/admin/expenses/{id}/

DELETE /api/admin/expenses/{id}/

POST /api/partners/expenses/

GET /api/partners/expenses/

Production Journal

GET /api/admin/production-journal/

POST /api/admin/production-journal/

GET /api/admin/production-journal/{id}/

Reports

GET /api/reports/admin-stats/

GET /api/reports/partner-stats/

GET /api/reports/store-history/

Geography

```
GET    /api/regions/

POST   /api/admin/regions/

PUT    /api/admin/regions/{id}/

DELETE /api/admin/regions/{id}/

GET    /api/cities/

POST   /api/admin/cities/

PUT    /api/admin/cities/{id}/

DELETE /api/admin/cities/{id}/
```

Messaging

```
GET    /api/chats/

GET    /api/chats/{id}/messages/

POST   /api/chats/{id}/messages/

DELETE /api/messages/{id}/

WS     /ws/chat/{room_id}/
```

Notifications

```
GET    /api/notifications/

POST   /api/notifications/mark-read/

DELETE /api/notifications/clear-all/

POST   /api/fcm-tokens/
```

20. МОДЕЛИ ДАННЫХ (ПОЛНЫЙ СПИСОК)

User

```
class User(AbstractUser):

    username = None # Отключаем

    phone = models.CharField(max_length=13, unique=True) # USERNAME_FIELD

    email = models.EmailField(unique=True)

    name = models.CharField(max_length=100)

    second_name = models.CharField(max_length=100)

    role = models.CharField(max_length=20, choices=ROLE_CHOICES)

    approval_status = models.CharField(max_length=20, default='pending')

    is_active = models.BooleanField(default=True)

    created_at = models.DateTimeField(auto_now_add=True)

    USERNAME_FIELD = 'phone'

    REQUIRED_FIELDS = ['email', 'name']
```

Product

```
class Product(models.Model):

    name = models.CharField(max_length=255, unique=True)

    price = models.DecimalField(max_digits=10, decimal_places=2)

    quantity = models.DecimalField(max_digits=10, decimal_places=2)

    is_weight_based = models.BooleanField(default=False)
```

```
is_bonus = models.BooleanField(default=False)

description = models.TextField(max_length=250)

is_active = models.BooleanField(default=True)

created_at = models.DateTimeField(auto_now_add=True)
```

Store

```
class Store(models.Model):

    name = models.CharField(max_length=255, unique=True)

    owner = models.ForeignKey(User, on_delete=models.CASCADE)

    owner_name = models.CharField(max_length=255)

    phone = models.CharField(max_length=13)

    inn = models.CharField(max_length=14, unique=True)

    region = models.ForeignKey(Region, on_delete=models.PROTECT)

    city = models.ForeignKey(City, on_delete=models.PROTECT)

    address = models.TextField()

    is_active = models.BooleanField(default=True)

    total_debt = models.DecimalField(max_digits=12, decimal_places=2,
default=0)

    paid_debt = models.DecimalField(max_digits=12, decimal_places=2,
default=0)

    created_at = models.DateTimeField(auto_now_add=True)
```

PartnerInventory

```

class PartnerInventory(models.Model):

    partner = models.ForeignKey(User, on_delete=models.CASCADE)

    product = models.ForeignKey(Product, on_delete=models.CASCADE)

    quantity = models.DecimalField(max_digits=10, decimal_places=2)

    weight = models.DecimalField(max_digits=10, decimal_places=2, null=True)

    partner_request_id = models.ForeignKey(PartnerRequest, null=True)

    class Meta:

        unique_together = ['partner', 'product']

```

StoreOrder

```

class StoreOrder(models.Model):

    store = models.ForeignKey(Store, on_delete=models.CASCADE)

    status = models.CharField(max_length=20, choices=ORDER_STATUS_CHOICES)

    order_type = models.CharField(max_length=20, choices=ORDER_TYPE_CHOICES)

    total_amount = models.DecimalField(max_digits=12, decimal_places=2)

    prepayment = models.DecimalField(max_digits=12, decimal_places=2,
default=0)

    debt_amount = models.DecimalField(max_digits=12, decimal_places=2,
default=0)

    bonus_count = models.IntegerField(default=0)

    defective_count = models.IntegerField(default=0)

    notes = models.TextField(blank=True)

    rejection_reason = models.TextField(blank=True)

```



```
created_at = models.DateTimeField(auto_now_add=True)

accepted_at = models.DateTimeField(null=True)

confirmed_by = models.ForeignKey(User, null=True,
on_delete=models.SET_NULL)
```

StoreInventory

```
class StoreInventory(models.Model):

    store = models.ForeignKey(Store, on_delete=models.CASCADE)

    product = models.ForeignKey(Product, on_delete=models.CASCADE)

    quantity = models.DecimalField(max_digits=10, decimal_places=2)

    weight = models.DecimalField(max_digits=10, decimal_places=2, null=True)

    bonus_count = models.IntegerField(default=0)

    paid_count = models.IntegerField(default=0)

    defective_count = models.IntegerField(default=0)

    class Meta:

        unique_together = ['store', 'product']
```

ReturnedItem

```
class ReturnedItem(models.Model):

    order = models.ForeignKey(StoreOrder, on_delete=models.CASCADE)

    product = models.ForeignKey(Product, on_delete=models.CASCADE)

    quantity = models.DecimalField(max_digits=10, decimal_places=2)

    weight = models.DecimalField(max_digits=10, decimal_places=2, null=True)
```

```
price = models.DecimalField(max_digits=10, decimal_places=2)

reason = models.TextField(blank=True)

returned_at = models.DateTimeField(auto_now_add=True)

returned_by = models.ForeignKey(User, on_delete=models.CASCADE)
```

ЗАКЛЮЧЕНИЕ

Это ****ФИНАЛЬНОЕ КОНСОЛИДИРОВАННОЕ ТЕХНИЧЕСКОЕ ЗАДАНИЕ БайЭл v3.0**** без противоречий.

****Документ основан на:****

-  Анализ всех технических заданий (v2.0 + v3.0)
-  Изучение текущего кода
-  Детальный анализ дизайна Figma (Admin + Partner + Store)
-  Система расходов из дизайна
-  Уточнения от клиента по всем критическим вопросам

****Ключевые решения v3.0:****

1.  Owner-based система магазинов
2.  Заказы магазина → Партнёр (напрямую, минуя админа)
3.  Партнёрский инвентарь (PartnerInventory + PartnerRequest)
4.  Два типа заказов (Предзаказ: PENDING→IN_TRANSIT→ACCEPTED / Ручной: сразу ACCEPTED)
5.  Возвращённые товары (ReturnedItem)
6.  Журнал производства (массовое добавление)
7.  Система расходов (Сюзерен/Вассал/Обыватель)

****Документ готов к разработке!**** 

****Общий объём:**** ~3500+ строк кода, детальные спецификации всех систем, примеры API, модели данных.

****Версия:** 3.0 Final**

****Дата:** 28 января 2026**

****Статус:** Утверждено к разработке**

****Автор:** Техническая команда БайЭл**